

DOSSIER PROJET

PROJET : WOOFWALKS

Rédacteur : Sarah Herberichs

Date : 07/10/2025

TABLE DES MATIERES

1. LISTE DES COMPETENCES DU REFERENTIEL COUVERTES PAR LE PROJET.....	4
1.1. Développer une application sécurisée.....	4
1.1.1. Installer et configurer son environnement de travail en fonction du projet.....	4
1.1.2. Développer des interfaces utilisateur.....	5
1.1.3. Développer des composants métier.....	6
1.1.4. Contribuer à la gestion d'un projet informatique.....	7
1.2. Concevoir et développer une application sécurisée organisée en couches.....	9
1.2.1. Analyser les besoins et maquetter une application.....	9
1.2.2. Définir l'architecture logicielle d'une application.....	9
1.2.3. Concevoir et mettre en place une base de données relationnelle.....	9
1.2.4. Développer des composants d'accès aux données SQL et NoSQL.....	10
1.3. Préparer le déploiement d'une application sécurisée.....	11
1.3.1. Préparer et exécuter les plans de tests d'une application.....	11
1.3.2. Préparer et documenter le déploiement d'une application.....	13
1.3.3. Contribuer à la mise en production dans une démarche DevOps.....	13
2. CAHIER DES CHARGES.....	14
2.1. Principes de référencement.....	14
2.2. Exigences de performances et de volumétrie.....	14
2.3. Multilinguisme & adaptations pour un public spécifique.....	14
2.4. Description graphique et ergonomique.....	15
2.4.1. Composants de la charte graphique.....	15
2.4.2. Design, Responsive design et autres exigences liées au design.....	16
2.5. Besoins fonctionnels « métier ».....	16
2.5.1. Utilisateurs du projet.....	16
2.5.2. Informations relatives aux contenus.....	16
2.5.3. Inventaire des besoins fonctionnels.....	17
2.5.4. Budget.....	21
3. PRÉSENTATION DE L'ENTREPRISE ET DU SERVICE.....	22
3.1. Présentation de l'entreprise et du service.....	22
3.2. Objectifs du projet.....	22
3.3. Cible adressée par le projet.....	23
4. GESTION DE PROJET.....	24
4.1. Intervenants sur le projet.....	24
4.2. Méthodologie.....	24
4.3. Outils, planning et suivi.....	24
4.4. Objectifs de qualité.....	26

5. SPÉCIFICATIONS FONCTIONNELLES.....	27
5.1. Contraintes du projet et livrables attendus.....	27
5.1.1. Criticité de l'application.....	27
5.1.2. Services tiers.....	27
5.1.3. Livrables attendus.....	27
5.2. Architecture logicielle du projet.....	28
5.3. Maquettes et enchaînement des maquettes.....	29
5.3.1. Cartographie.....	29
5.3.2. Maquettes.....	30
5.4. Modèle entités-associations et modèle physique de la base de données.....	34
5.4.1. MCD (MERISE).....	34
5.4.2. MLD (MERISE).....	35
5.4.3. Dictionnaire de données.....	36
5.4.4. MPD (MERISE).....	37
5.4.5. Diagramme de classes (UML).....	38
5.5. Script de création et/ou de modification de la base de données.....	39
5.5.1. Script de création.....	39
5.5.2. Script de modification.....	40
5.6. Diagramme du comportement des fonctionnalités de type cas d'utilisations.....	41
5.6.1. Diagramme de cas d'utilisation (UML).....	41
5.7. Fonctionnalités détaillées les plus significatives.....	42
5.7.1. Fonctionnalité n° 1 : S'inscrire à une balade.....	42
5.7.2. Fonctionnalité 2 : Poster une annonce pour une balade.....	47
5.7.3. Fonctionnalité 3 – Commander un Album Photo.....	51
5.7.4. Fonctionnalité 4 – Invitation à poster un feedback.....	55
5.7.5. Fonctionnalité 5 – Poster un Feedback.....	59
6. SPÉCIFICATIONS TECHNIQUES.....	61
6.1. Référencement.....	61
Environnement technique.....	62
Navigation et accessibilité :.....	63
Services tiers :.....	63
6.2. Sécurité.....	64
7. RÉALISATIONS.....	65
7.1. Exemple 1 : Poster une annonce pour Balade (Composants Métier).....	65
7.1.1. Affichage : Cf.Diagrammes.....	65
7.1.2. Extrait(s) de code.....	65
7.1.3. Argumentation :.....	70
7.2. Exemple 2 : Enregistrement d'un Utilisateur (Composants Métier).....	71

7.2.1. Affichage.....	71
7.2.2. Extrait(s) de code.....	72
7.2.3. Argumentation :.....	76
7.3. Exemple 3 : (Composants d'accès aux données) : Voir Ci-dessus, utilisation des Repositories et de l' entity manager.....	77
8. ÉLÉMENTS DE SÉCURITÉ DE L'APPLICATION.....	78
9. PLAN DE TESTS.....	83
10. JEU D'ESSAI DE LA FONCTIONNALITÉ LA PLUS REPRÉSENTATIVE.....	84
10.1. Fonctionnalité testée.....	84
10.2. Description des scenarios.....	84
10.3. Résultats des tests.....	84
10.4. Conclusion.....	85
Les tests de performance que j'ai réalisé sur l'application ont été concluants et ont atteint les objectifs définis. J'ai pu valider la stabilité du système.....	85
11. VEILLE SUR LES VULNÉRABILITÉS DE SÉCURITÉ.....	86

1. LISTE DES COMPETENCES DU REFERENTIEL COUVERTES PAR LE PROJET

1.1. Développer une application sécurisée

1.1.1. Installer et configurer son environnement de travail en fonction du projet

Pour ce projet, j'ai mis en place mon environnement de travail afin qu'il corresponde aux besoins techniques du projet. J'ai installé et configuré **Docker** pour gérer les conteneurs et garantir une cohérence entre les environnements de développement et de production. J'ai également configuré **Symfony avec API Platform** pour la partie backend, en m'assurant que l'API puisse gérer correctement les entités et les routes nécessaires. Pour la partie frontend, j'ai installé **React** et configuré l'interaction avec l'API, en mettant en place les dépendances nécessaires via npm et en adaptant la structure des dossiers pour un développement clair et modulable. J'ai ainsi créé un environnement permettant de développer, tester et déployer le projet efficacement.

Outils et technologies :

- **VS Code** : IDE principal
- **Docker**: orchestration des services backend, frontend et base de données
- **Symfony + API Platform** : backend API REST sécurisée
- **React** : frontend SPA (Single Page Application)
- **MySQL** : base de données relationnelle
- **phpMyAdmin** : interface graphique pour gérer la base
- **Nginx** : serveur web pour le backend et reverse proxy

Configuration mise en place :

- Si l'utilisateur n'est pas connecté :
 - L'utilisateur est invité à se connecter
- Les détails de l'annonce n'ont pas été trouvés en BDD
 - Le système affiche un message d'erreur
- Si l'utilisateur demande à recevoir une notification si une place se libère, mais que la procédure échoue
 - Le système affiche un message d'erreur
- Si l'utilisateur demande à rejoindre la balade, mais que la sauvegarde échoue :
 - Le système affiche un message d'erreur
- Si le système ne trouve pas les détails relatifs à l'affichage de la version participant de la balade

Sécurité et variables d'environnement :

1- Gestion par fichiers .env selon le contexte :

- .env : configuration générale
- .env.local : configuration locale
- .env.test : environnement de test avec base dédiée.
- .env React : URL de l'API .
- .env.docker : configuration des identifiants d'accès à la BDD pour Docker

- Couplé a un **Makefile** pour simplifier les commandes Docker de base

2- Dossier security.yaml :

- **password_hashers** → comment sont hachés les mots de passe
- **providers** → comment Symfony charge les utilisateurs
- **firewalls** → quelles parties de l'appli sont protégées et comment on s'authentifie
- **access_control** → qui a accès à quelles routes (droits et rôles)

3- Certificats SSL :

Pour chiffrer les échanges et simuler un environnement HTTPS de bout en bout, j'ai généré des certificats pour le frontend et le backend. Ces certificats ont été intégrés dans la configuration Docker Compose afin que le protocole HTTPS soit actif et pris en compte dans tous les services.

4- Gitignore :

- Exclusion des fichiers sensibles dans gitignore
- **Récupération des Logs**

Avantages de cette configuration :

Portabilité : une commande simple pour lancer tout le projet

Isolation : chaque service tourne dans son conteneur

Sécurité : HTTPS et gestion stricte des cookies JWT même en local

1.1.2. Développer des interfaces utilisateur

Conformité au dossier de conception et aux maquettes Figma :

- L'interface React est conforme aux maquettes et au dossier de conception
- Les composants sont réutilisables avec des nuances adaptées aux besoins spécifiques (ex : formulaire de post de balades « injecté » dans le formulaire générique)

Adaptation au type d'utilisation et fluidité :

- Les composants React sont fluides et réactifs, optimisés pour un usage intensif
- L'interface s'adapte à différents écrans (responsive design) grâce aux styles CSS et Bootstrap

Documentation et tests :

- Le code est documenté en français et en anglais
- Des tests unitaires ont été mis en place et effectués avec JEST
- Les cas d'usage principaux ont été pris en compte dans la phase de développement

Démarche de résolution de problèmes :

- En cas d'erreur détectée lors des tests, une démarche structurée est appliquée : identification du bug → isolation du composant → correction → re-test

Sécurité de l'interface et des API

Sécurité front React :

- L'utilisation du **JSX** empêche l'injection directe de HTML dans le rendu et **DOMPurify** pour nettoyer les entrées en entrée
- La gestion d'état et des props encadre les interactions utilisateur et limite les risques d'injections de données
- **Validation systématique des entrées** et gestion des erreurs améliorent la robustesse de l'interface

Sécurisation des API REST et backend :

- **Authentification et autorisation** : JWT pour vérifier que chaque requête est authentifiée. Les rôles et permissions côté serveur contrôlent l'accès aux endpoints sensibles
- **HTTPS** : toutes les communications sont chiffrées pour empêcher l'interception et la modification des données
- **Validation côté serveur** : filtrage et validation des données indépendamment des validations front

Protection contre les principales failles :

- Cross Site Scripting XSS : Paramétrage des **CSP** pour limiter les contenus autorisés (Protection contre le ClickJacking, limitation du contenu présent seulement au domaine de woofwalks)
- Injection SQL/NoSQL : utilisation de requêtes doctrine
- Rate limiting : pour éviter les abus et se protéger du brute force. (rate_limiter)

Sécurisation des données sensibles : tokens et mots de passe jamais stockés en clairs (utilisation du password_hasher **PasswordAuthenticatedUserInterface** symfony) et de cookies sécurisés et HttpOnly)

Accessibilité et charte graphique :

- Respect du RGAA (Référentiel Général d'Amélioration de l'Accessibilité) pour les interfaces web
- Charte graphique appliquée à tous les composants et respect des normes ergonomiques UX/UI : simplicité, lisibilité, minimalisme

Normes et bonnes pratiques techniques :

- Validation systématique des entrées utilisateurs, gestion complète des exceptions et erreurs pour éviter les vulnérabilités
- Respect des normes de codage React/JSX
- Utilisation de composants graphiques réutilisables
- Documentation technique et code en Anglais

1.1.3. Développer des composants métier

Dans le développement des composants métier de l'application, j'ai appliqué les **principes SOLID** et un style défensif pour garantir la maintenabilité et la sécurité du code. Les composants React, tels que PostGenericAd, sont conçus pour injecter dynamiquement des formulaires spécifiques (PostWalkForm, PostParkForm, PostHikeForm etc.), et ils font appel à des **services métiers asynchrones** qui centralisent la logique métier côté Symfony. Chaque service géré sur Symfony suit une **architecture en couches**, séparant clairement la gestion des données, la logique métier et la communication avec l'API.

Pour la **sécurité et la gestion des comptes utilisateurs** :

- Lors de la **création de compte**, le **mot de passe est hashé** avec un algorithme sécurisé
- Un **email de confirmation contenant un token** est envoyé à l'utilisateur, et le compte n'est activé que si l'utilisateur valide ce mail
- L'authentification repose sur un **système de tokens JWT** :
 - JWT à courte et longue durée et JWT longue durée servant de preuve de bonne identification
 - Contrôle automatique et Refresh si nécessaire du JWT court lors de chaque action si le long est encore valide
 - les JWT sont **stockés dans les cookies HttpOnly et Secure**, empêchant l'accès côté client et limitant les risques XSS
- Un **contrôle d'accès centralisé** s'applique sur tous les services métiers pour gérer droits et restrictions d'accès

Côté **tests**, j'ai mis en place :

- des **tests unitaires Symfony** avec **PHPUnit** pour valider chaque service métier et repository
- des **tests fonctionnels et d'intégration frontend** avec **Jest** pour vérifier la réactivité des composants React et le comportement des formulaires dynamiques
- une **intégration continue** via **GitHub Actions** sur chaque push vers **main**, exécutant les tests et générant des rapports
- des **tests de sécurité automatisés** via **ZAP**, assurant la détection des vulnérabilités potentielles dans l'application

Le code respecte les **naming conventions Symfony** et est documenté en français et en anglais. Les dysfonctionnements détectés par les tests sont analysés avec une **démarche structurée de résolution de problème**, permettant de corriger rapidement et de manière fiable tout bug ou faille.

L'ensemble respecte les Principe **S.O.L.I.D**

1.1.4. Contribuer à la gestion d'un projet informatique

Dans ce dossier, on peut passer en revue l'ensemble de ma gestion d'un projet informatique. La réflexion a commencé par la rédaction littéraire de ma visualisation de l'application. J'ai ensuite réalisé un **inventaire des besoins fonctionnels** et des **User Stories**, que j'ai alimenté au fur et à mesure en précisant les besoins principaux et les tâches nécessaires à leur réalisation. À partir de ces éléments, j'ai établi progressivement des **cards** détaillés pour chaque fonctionnalité sur **Trello**, organisées par ordre de priorité afin de viser rapidement un **MVP**.

En parallèle, j'ai réalisé une cartographie du projet, des maquettes ainsi que l'ensemble de la modélisation **Merise** et **UML**. Pour chaque élément complexe, j'ai produit des diagrammes détaillés afin de bien prendre en compte les subtilités chronologiques et opérationnelles des implémentations en cours ou à venir. Ces modélisations ont été directement liées aux composants développés, notamment les **entités**, les **repositories** et les **services**, ce qui a permis de traduire la planification en développement concret et structuré.

Concernant la qualité du code et du suivi, j'ai mis en place des **tests unitaires**, effectué régulièrement du **refactoring** et commenté mon code pour assurer sa cohérence et sa **maintenabilité**.

L'environnement de développement choisi était adapté à l'architecture du projet : **Symfony et API Platform** pour la partie backend, **SQL** pour la base de données, et **React** pour le frontend. Symfony a été sélectionné pour sa simplicité d'usage et sa bonne compatibilité avec React, tandis que React a permis de créer une application fluide et modulable grâce à son découpage en composants réutilisables. **GitHub** a été utilisé pour le **versioning**, avec une **branche spécifique pour chaque fonctionnalité majeure**. Chaque validation finale était mergée sur la branche principale **develop**.

Concernant les **outils collaboratifs** et la **méthode de gestion**, j'ai suivi une approche **Agile** en particulier inspiré de la méthode **SCRUM** en découplant le projet en **itérations** et en suivant mes cards Trello, en m'assurant que les priorités étaient respectées et que l'avancement restait conforme aux objectifs définis.

J'ai pris le temps de vérifier régulièrement la cohérence de chaque tâche avec le besoin initial et avec le planning défini avant de lancer leur réalisation. Même en l'absence de réunions avec un client, ce suivi m'a permis de m'assurer que les fonctionnalités développées répondaient bien aux objectifs du projet et respectaient l'ordre de priorité défini dans mes **user stories**.

Chaque weekend, j'établissais un plan d'action pour identifier les tâches en retard ou nécessitant un ajustement. Certaines tâches prenant plus de temps que prévu ont entraîné des décalages, que j'ai rattrapés en réorganisant mon emploi du temps et en allouant des périodes supplémentaires pour respecter les délais.

En résumé, ce projet a combiné planification rigoureuse, suivi précis, qualité du code, documentation structurée et adéquation de l'environnement technique avec les besoins fonctionnels, tout en appliquant une méthodologie inspirée de **SCRUM** et des outils collaboratifs pour garantir un développement efficace et évolutif.

1.2. Concevoir et développer une application sécurisée organisée en couches

Comme vu plus haut, mon application a été découpé sur les principes de **SOLID**, dans un but d' **ouverture à la modification et d'ajout de nouvelles entités**, **ré-utilisabilité**, **scalabilité**, confort à la compréhension et allègement du code.

1.2.1. Analyser les besoins et maquetter une application

Après analyse des besoins de l'application, j'ai mis en place une **modélisation** permettant d'anticiper l'architecture nécessaire. J'ai ensuite réalisé une **maquette** interactive sur **Figma**. J'ai porté une attention particulière à la conception des parcours utilisateurs sensibles comme l'authentification et la sécurisation des routes, en définissant un système basé sur des JWT stockés dans les cookies, renouvelés toutes les 15 minutes. Cette partie a été schématisée afin d'identifier clairement les points critiques et d'anticiper les éventuelles erreurs.

Dans une démarche d'accessibilité, j'ai également vérifié mes choix de couleurs et de typographies au regard du **RGAA**, en testant les contrastes et la lisibilité pour garantir un affichage conforme aux règles d'accessibilité.

1.2.2. Définir l'architecture logicielle d'une application

Dans le cadre de ce projet, l'architecture logicielle a été conçue selon le modèle **MVC (Model-View-Controller)** proposé par Symfony. Les **contrôleurs** gèrent les requêtes des utilisateurs et coordonnent les actions, les **modèles (entités)** représentent les données et la logique métier associée, et les **vues REACT** s'occupent de l'affichage des informations.

Pour organiser l'accès aux données, j'ai utilisé le **Repository Pattern**, qui centralise toutes les opérations sur la base de données et permet de séparer la logique d'accès aux données du reste de l'application. La logique métier plus complexe est encapsulée dans des **services**, suivant le **Service Layer Pattern**, afin de favoriser la ré-utilisabilité et la testabilité du code.

Les **interfaces** ont été utilisées pour définir des contrats précis entre différentes parties de l'application, appliquant le principe de **Dependency Inversion**, ce qui permet de changer facilement l'implémentation d'un service ou d'un composant sans impacter le reste du code.

Pour les fonctionnalités plus simples et standardisées, notamment les opérations **CRUD** basiques, j'ai utilisé **API Platform**, ce qui a permis de générer rapidement les endpoints REST et de gérer la validation et la sérialisation automatiquement, tout en restant compatible avec l'architecture multicouche.

Cette combinaison de patterns et d'outils garantit une architecture **multicouche**, **modulaire**, **sécurisée**, conforme aux bonnes pratiques, tout en facilitant la **maintenance**, l'**évolutivité** et le **développement rapide** des fonctionnalités.

1.2.3. Concevoir et mettre en place une base de données relationnelle

J'ai commencé par analyser les besoins exprimés dans le cahier des charges. Cela m'a permis de concevoir le modèle conceptuel des données, en identifiant toutes les entités nécessaires, leurs attributs et les relations entre elles. J'ai utilisé les diagrammes Merise pour représenter clairement ces relations et optimiser l'organisation des données, en respectant les règles relationnelles et de nommage en vigueur.

J'ai produit un **modèle conceptuel de données (MCD)** puis le **modèle logique des données (MLD)** en traduisant les entités et relations en tables, attributs et clés primaires/étrangères. J'ai ensuite créé le **modèle physique des données (MPD)**.

J'ai créé un jeu de données complet dans une base de test, couvrant tous les cas d'utilisation. J'ai utilisé les **fixtures Symfony** pour peupler la base.

J'ai **configuré la connexion à la base de données** dans le fichier `.env` et protégé les informations sensibles via `.gitignore`. J'ai ensuite créé la base avec la commande `doctrine:database:create`.

Dans Docker, j'ai configuré un conteneur MySQL et un volume pour assurer la **persistance des données** même si le conteneur est supprimé.

J'ai défini les **contraintes** directement dans les entités Symfony pour optimiser l'accès aux données et assurer l'intégrité du système. J'ai également mis en place des relations entre entités (OneToMany,ManyToOne, ManyToMany) pour refléter le modèle conceptuel.

Tout au long de ce processus, j'ai appliqué mes connaissances sur les bases relationnelles, le SQL, les codages de données, et les bonnes pratiques de sécurisation.

1.2.4. Développer des composants d'accès aux données SQL et NoSQL

Dans le cadre de mon projet, j'ai développé les traitements permettant la consultation, la création, la modification et la suppression des données. Les opérations critiques sont conçues pour rester **robustes face aux erreurs**. Par exemple, les **cas d'exception**, tels que la tentative de suppression d'une donnée inexistante, sont pris en compte : Symfony et API Platform génèrent automatiquement des réponses HTTP appropriées (par exemple 404 Not Found), ce qui permet à l'application de rester stable sans planter.

Certaines fonctionnalités **CRUD** sont **couverts par des tests unitaires**, afin de vérifier les droits d'accès et la résistance aux entrées malveillantes.

En cas de dysfonctionnement, j'applique une **démarche structurée de résolution de problème**, analysant les logs et les erreurs pour identifier la cause et corriger le problème.

Sur le plan technique, j'ai :

- Mis en place la gestion de l'intégrité des données en base, en assurant que chaque enregistrement possède un **clé primaire unique**, que les relations entre tables restent cohérentes via des **clés étrangères**, et que les valeurs respectent les **règles métier définies** (types, plages, formats). J'ai mis en place et vérifié ces règles à la fois par des contraintes SQL et des validations côté application.
- Implémenté de façon sécurisée les accès aux données relationnelles
- Programmé des fonctions dans un style défensif, en validant systématiquement toutes les entrées.
- Connu et appliqué les bonnes pratiques contre les principales attaques sur les bases de données (injection SQL, force brute, Cross Site Scripting (XSS), élévation de privilèges) en utilisant :
 - Les échappements dus à l'utilisation du **JSX**, permettant de se protéger des **XSS**
 - **Doctrine** et ses requêtes préparées
 - La gestion des **rôles** Symfony pour éviter l'élévation de privilège
- Compris les principes de fonctionnement des **transactions**, de leurs niveaux d'isolation et du verrouillage des données, même si je ne les ai pas implémentées directement dans ce projet
- Planifié et exécuté des tests unitaires et des tests de sécurité sur les fonctionnalités critiques
- Utilisé une base SQL, car l'environnement Symfony/React s'y prête naturellement et facilite la gestion des entités et des relations. Dans un autre projet, une base NoSQL aurait permis de gérer plus facilement des données très volumineuses ou hétérogènes, avec une structure flexible et évolutive
- J'ai également intégré des **outils d'analyse statique de code** comme **PHPStan** qui m'a permis de détecter des morceaux de code mort, vérifier le typage de mes fonctions et m'alerter sur des problèmes qui auraient pu être problématique par la suite (ordre d'arguments de fonctions par exemple)

Ce projet m'a permis de mettre en œuvre à la fois mes savoir-faire techniques, organisationnels, en garantissant un système robuste, sécurisé et maintenable.

1.3. Préparer le déploiement d'une application sécurisée

1.3.1. Préparer et exécuter les plans de tests d'une application

Pour mettre en place les tests, j'ai commencé par organiser mon environnement, j'ai tout d'abord installé PHPUnit.

Pour les tests fonctionnels :

Création de la BDD de tests

```
sarah@LAPTOP-56Q9QUJ6:~/woofwalks_project$ docker exec -it woofwalks_php-fpm bash
root@12b9c461d892:/var/www/html# php bin/console doctrine:database:create --env=test
Created database `woofwalks_test_test` for connection named default
root@12b9c461d892:/var/www/html# php bin/console doctrine:schema:create --env=test

!
! [CAUTION] This operation should not be executed in a production environment!
!

Creating database schema...

[OK] Database schema created successfully!
```

Création de data fixtures pour peupler la BDD test :

- Installation de doctrine-fixtures-bundle
- Création de fichiers fixtures
- Chargement des fixtures créées dans le fichier :
`php bin/console doctrine:fixtures:load --env=test --no-interaction`
- Implémentation et lancement des tests :
`APP_ENV=test ./vendor/bin/phpunit tests/Controller/WalkControllerFunctionalTest.php`

TEST ZAP OWASP :

Les tests OWASP ZAP ont révélé des problèmes liés à la **CSP (Content Security Policy)**, principalement dus au fait que l'application était en **environnement de développement**

→ Nécessité d'implémenter le mode production

J'ai relancé les scans après être passé en **mode production**, ce qui a permis de réduire certains faux positifs :

- Installation et configuration de **NelmioSecurityBundle** pour gérer les CSP et renforcer les headers de sécurité
- Création d'un fichier `web_profiler.yaml` pour désactiver le profiler en prod
- **Configuration Nginx :**
 - Modification de `default.conf` pour ajouter des **headers de sécurité après scan ZAP OWASP** qui m'a permis de les détecter
 - Ajout de directives CORS pour le front React
- **Masquage de la version PHP dans le php.ini**

1.3.2. Préparer et documenter le déploiement d'une application

J'ai préparé le déploiement de l'application, en suivant une approche **DevOps** qui met l'accent sur l'automatisation et la collaboration entre le développement et l'exploitation.

Outils et processus :

1. L'automatisation via GitHub Actions :

J'ai configuré un workflow d'**intégration continue (CI)** qui se déclenche automatiquement à chaque push sur la branche `main`

Ce workflow exécute une série de tâches : il prépare un environnement de test isolé, installe les dépendances, puis lance l'ensemble des tests automatisés

Mon rôle : M'assurer que le code est stable et fonctionnel avant d'être fusionné et déployé, ce qui permet de détecter les bugs et les failles de sécurité au plus tôt

2. L'infrastructure de production :

J'ai préparé une architecture conteneurisée à l'aide de **Docker Compose** pour la production. Cela garantit que l'environnement de l'application est cohérent et reproductible, quel que soit le serveur

Mon architecture utilise un **serveur web Nginx** qui sert à la fois le Frontend React compilé et agit comme un reverse proxy pour rediriger les requêtes de l'API vers le Backend Symfony

Mon rôle : Configurer une infrastructure sécurisée et performante, avec un point d'entrée unique et optimisé pour servir les fichiers statiques rapidement

1.3.3. Contribuer à la mise en production dans une démarche DevOps

Un environnement de production a été préparé avec un fichier `docker-compose.prod.yml` qui orchestre les services (nginx, php-fpm, base MySQL, frontend compilé). La configuration est sécurisée (désactivation des logs et du mode debug, variables adaptées à la prod). L'architecture est conçue pour servir l'application complète via une seule URL publique, nginx jouant le rôle de reverse proxy.

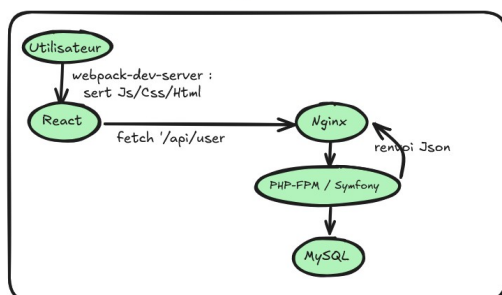
```
volumes:
- ./woofwalks_back:/var/www/html:ro
- ./woofwalks_back/docker/nginx/default.conf:/etc/nginx/conf.d/default.conf
- ./certs:/etc/nginx/certs:ro
- ./woofwalks_front/build:/usr/share/nginx/html:ro # Nginx sert le front compilé
```

```
frontend:
  build:
    context: ./woofwalks_front
    dockerfile: Dockerfile.prod
  container_name: woofwalks_front
  working_dir: /app
  command: ["npm", "run", "build"]
```

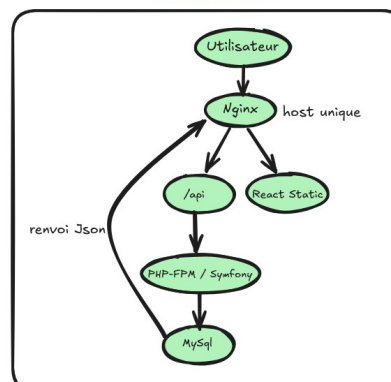
```
environment:
- APP_ENV=prod
- APP_DEBUG=0
depends_on:
```

```
healthcheck:
  test: ["CMD-SHELL", "mysql -uroot -p${MYSQL_ROOT_PASSWORD} -e 'SELECT 1;'"]
```

environnement développement : NPM start



environnement production :
NPM run build = compilation
dossier build



2. CAHIER DES CHARGES

2.1. Principes de référencement

Le site doit être optimisé pour des **mots-clés locaux** comme "*balades canines à [ville]*" ou "*parc pour chiens à [région]*". Le contenu doit respecter les bonnes pratiques SEO : titres pertinents, méta descriptions bien rédigées, et utilisation réfléchie des mots-clés. L'optimisation **mobile** (responsive design) et la **vitesse de chargement** sont essentielles pour un bon référencement et une meilleure expérience utilisateur.

2.2. Exigences de performances et de volumétrie

Nombre de visiteurs escomptés par mois : 200 à 500 visites par mois

Nombre de prospects collectés par les formulaires du site par mois

Nombre de nouveaux clients escomptés par mois : 50

Capacité de gestion de photos : gestion de 10 à 50 photos par rubrique feedback d'une balade

2.3. Multilinguisme & adaptations pour un public spécifique

Le site, bien que destiné uniquement à un public français, doit intégrer les bases de l'accessibilité numérique, notamment pour :

- Les malvoyants
- Les personnes dyslexiques
- Les utilisateurs âgés ou en situation de handicap cognitif ou moteur.

Voici la liste des principes à respecter :

Accessibilité visuelle :

- Police à sans serif, minimum 16px
- Contraste élevé
- Structure claire : titres hiérarchisés et textes bien espacés
- Navigation possible au clavier

Pour la dyslexie :

- Utilisation de phrases courtes et simples
- Pas de texte justifié
- Pas de majuscules en bloc
- Utilisation d'une police adaptée : **Lexend**, Open Sans, Robot
- Pas de couleurs proches
- Interlettrage suffisant
- Interligne suffisante
- Taille de police d'au moins 16px
- Espacement tactile assez large pour faciliter la navigation (boutons et liens)

Langue & compréhension :

- Site en français clair
- Balise HTML : `<html lang="fr">` pour les lecteurs d'écran

2.4. Description graphique et ergonomique

2.4.1. Composants de la charte graphique

Identité visuelle :

L'identité visuelle repose sur plusieurs éléments complémentaires, déjà définis :

Le **logo**, garant de la reconnaissance de la marque sur toutes les pages, est utilisé sur le header de chaque page



La **palette de couleurs** a été soigneusement choisie et testée selon les normes WCAG pour garantir un contraste suffisant. Les couleurs sont réparties fonctionnellement dans l'application, assurant une cohérence visuelle, une navigation intuitive et permettant la **navigation sans difficulté à tout public**

Couleurs :

Element	Couleur	Vue
Fond	Blanc-gris	#D9D9D9
Boutons, Menu Burger, Header	Vert forêt	#1B5E20
Texte boutons	Blanc cassé	#F5F5F5
Logo et Bouton Burger	Blanc	#FFFFFF
Texte général	Noir	#000000
Annulations, Erreurs	Rouge	#FF0000

Les polices de caractères :

La font préférée choisie a été **Lexend**, conçu spécialement pour sa facilité de lecture, adapté aux personnes ayant des difficultés de lecture.

Voici un exemple de police Lexend, premier choix paramétré en CSS pour des raisons de lisibilité

Le bandeau supérieur, les Popups, les Fond de l'application et son texte principal sont testées au **WCAG** avec une note minimale **AA**.

Taille des caractères selon les supports :

Pour assurer une lisibilité optimale tout en respectant les normes d'accessibilité (notamment un minimum de 16px pour le texte de corps), les tailles des caractères sont définies ainsi :

Élément	Desktop	Tablette	Mobile
Texte de corps	16px – 18px	16px	16px
Titres principaux (h1)	32px – 36px	28px – 32px	24px – 28px
Titres secondaires (h2, h3)	24px – 28px	20px – 24px	18px
Boutons et liens	18px - 20px	18px -20px	20px

2.4.2. Design, Responsive design et autres exigences liées au design

Le site doit être adapté à la lecture sur Desktop, tablette et mobile.

2.5. Besoins fonctionnels « métier »

2.5.1. Utilisateurs du projet

Présentation de la répartition des droits des utilisateurs

Visiteur :

Accède aux pages publiques, peut visualiser la liste des annonces mais sans accès au détail complet. Il a la possibilité de créer un compte pour accéder à plus de fonctionnalités.

Utilisateur :

- Consulter les informations détaillées d'une balade
- S'inscrire à une balade
- Recevoir des notifications personnalisées
- Participer au chat dédié aux balades auxquelles il est inscrit
- Poster des feedbacks
- Accéder aux galeries photos des balades auxquelles il a participé
- Commander un album photo personnalisé

Administrateur du site : Supervise et modère tous les contenus publiés, gère les comptes utilisateurs, et veille au respect de la charte et des règles du site. Il assure également la maintenance et la sécurité de la plateforme.

2.5.2. Informations relatives aux contenus

Contenu accessible :

L'utilisateur peut consulter des listes d'annonces, chacune comportant une image principale en plus des informations générales relatives à l'événement. Lorsqu'il accède aux feedbacks d'une balade à laquelle il a participé, il peut visualiser toutes les images postées par les autres participants ainsi que participer au chat dédié à cette balade.

Droit à l'image :

Conformément au **RGPD**, la participation à une balade implique une acceptation tacite de la publication des contenus (photos, messages) où le participant apparaît. Lors de la création de son compte, l'utilisateur doit accepter explicitement les conditions liées au RGPD en cochant une case obligatoire prévue à cet effet.

2.5.3. Inventaire des besoins fonctionnels

Thème	Qui	Quoi	Pourquoi
Compte	Visiteur	M'inscrire	Accéder aux fonctionnalités de l'application et participer aux balades
Authentification	Utilisateur	Me connecter	Utiliser la totalité des fonctionnalités de l'application
Authentification	Utilisateur	Me déconnecter	Quitter l'application
Compte	Utilisateur	Modifier mes informations de profil	Mettre à jour mes données personnelles et afficher les bonnes informations aux autres utilisateurs
Localisation	Utilisateur	Modifier ma localisation à un instant t	Adapter les suggestions de balades à ma position actuelle
Compte	Utilisateur	Supprimer mon compte	Quitter la plateforme et effacer mes données personnelles
Notifications	Utilisateur	Choisir mes préférences de notifications	Recevoir uniquement les informations qui m'intéressent
Animal	Utilisateur	Ajouter les informations sur mon chien	Pour que les autres utilisateurs puissent avoir une idée de la taille, comportement, tempérament...
Contact	Visiteur Utilisateur	Contactar les administrateurs de l'application	Communiquer avec l'entreprise, pour toute information
Modération	Utilisateur	Signaler un utilisateur ou du contenu non approprié ou contraire aux RGPD	Enclencher un contrôle afin de garantir un espace sûr
Annonce	Utilisateur	Créer une annonce	Préparer une annonce pour sortir mon chien avec d'autres maîtres, se sentir moins seul, partager...
Annonce	Utilisateur	Poster une annonce	Publier définitivement l'annonce créée
Annonce	Utilisateur	Modifier une annonce	Ajuster l'annonce, prévenir les inscrits d'un changement grâce à une alerte
Annonce	Utilisateur	Supprimer une annonce	Empêcher de nouvelles inscriptions et prévenir les inscrits de l'annulation via une alerte
Annonce	Utilisateur	Poster une copie d'une ancienne annonce	Gagner du temps si j'organise souvent les mêmes balades
Lieu	Utilisateur	Créer une annonce dans un lieu référencé comme parc	Proposer un lieu connu et adapté aux chiens
Lieu	Utilisateur	Créer une annonce dans un lieu customisé	Organiser une balade dans un endroit moins connu ou privé

Thème	Qui	Quoi	Pourquoi
Annonces	Utilisateur	Afficher toutes les annonces autour de moi	Visualiser rapidement l'ensemble des balades prévues susceptibles de me correspondre
Annonces	Utilisateur	Filtrer les annonces selon mes critères	Trouver plus rapidement une balade qui correspond à mes envies
Annonces	Utilisateur	Trier les annonces selon mes critères	Trouver plus rapidement une balade qui correspond à mes envies
Participants	Utilisateur	Consulter le profil des participants et de leurs chiens	Évaluer la compatibilité entre les animaux, voir si cela me correspond
Feedbacks	Utilisateur	Consulter les textes des feedbacks des anciennes balades de l'organisateur	Évaluer la qualité des événements passés et savoir si cela correspond à mes attentes
Annonces	Utilisateur	Voir les détails d'une annonce	Savoir précisément où, quand et comment elle se déroule
Balades	Utilisateur	Voir qui est inscrit à la balade	Avoir une idée des participants et de l'ambiance prévue
Balades	Utilisateur	Voir les balades prévues sur une carte	Visualiser rapidement où elles ont lieu
Balades	Utilisateur	M'inscrire à une balade	Participer à une balade qui me correspond
Balades	Utilisateur	Annuler mon inscription	Libérer ma place et permettre à quelqu'un d'autre de s'inscrire
Notifications	Utilisateur	Demander à recevoir des notifications si une place se libère	Être alerté pour pouvoir m'inscrire dès qu'une place est disponible
Album	Utilisateur	Sélectionner des photos des balades auxquelles j'ai assisté pour créer un album	Sélectionner celles qui me plaisent le plus
Album	Système	Valider l'achat de mon album photo	Recevoir l'album que j'ai créé
Chat	Utilisateur	Accéder au chat de la balade dès inscription	Échanger avec les autres participants de la balade
Chat	Utilisateur	Envoyer des messages aux utilisateurs inscrits dans la balade	Échanger avec les autres participants avant la balade
Chat	Utilisateur	Recevoir des messages des utilisateurs de la balade	Être informé en direct et pouvoir réagir
Notifications	Utilisateur	Être notifié des changements de la balade	Être immédiatement au courant des évolutions
Notifications	Utilisateur	Être notifié des messages du chat de la balade	Rester informé des échanges entre participants

Thème	Qui	Quoi	Pourquoi
Notifications	Utilisateur	Être notifié lorsqu'une place se libère dans une balade	Pouvoir m'inscrire avant les autres
Feedbacks	Utilisateur	Envoyer des feedbacks sur une balade avec photos, texte	Partager mon expérience et alimenter la banque de photos
Feedbacks	Utilisateur	Consulter les feedbacks d'une balade passée	Revoir les retours des autres et revivre l'événement
Administration	Administrateur	Consulter toutes les annonces	Surveiller leur conformité
Administration	Administrateur	Supprimer, modifier, masquer une annonce	Garder une application propre et conforme
Modération	Administrateur	Recevoir les signalements utilisateurs	Réguler le contenu inapproprié
Administration	Administrateur	Accéder et modifier les profils	Répondre aux demandes ou bannir si nécessaire
Modération	Administrateur	Supprimer un compte utilisateur	Répondre à une demande ou sanctionner un non-respect des CGU
Modération	Modérateur	Modérer les messages des chats	Réguler le contenu inapproprié
Modération	Modérateur	Modérer les feedbacks	Réguler le contenu inapproprié

2.5.4. Budget

Le budget prévisionnel du projet a été établi sur la base d'un **découpage fonctionnel** du site, réparti en trois niveaux de priorité :

Niveau 1 : fonctionnalités indispensables au bon fonctionnement de l'application

Niveau 2 : fonctionnalités importantes mais non vitales

Niveau 3 : fonctionnalités complémentaires ou d'optimisation

Chaque niveau a fait l'objet d'une **estimation du temps de développement et a permis de préétablir un budget**, prenant en compte :

- La **complexité technique** de chaque fonctionnalité
- Un **coût journalier** basé sur un tarif de 180 € (correspondant à mon activité d'autoentrepreneur)
- Des pourcentages complémentaires appliqués sur le temps de développement :
 - Conception : 20 %
 - Tests : 10 %
 - Gestion de projet : 0,5 %
 - Marge pour imprévus : 15 %

Ajustement et priorisation :

Le **budget global estimé à 22 950 €** (intégrant tous les niveaux avec marges), dépassant potentiellement le budget cible du client.

Ce constat permet d'envisager différentes options :

- **Prioriser les niveaux 1 et 2** pour respecter le budget
- Ou **répartir le développement en plusieurs phases** dans le temps

Cette approche démontre ma capacité à **chiffrer un projet de manière réaliste** et à proposer des **solutions d'ajustement budgétaire** en fonction des objectifs et des contraintes du commanditaire.

3. PRÉSENTATION DE L'ENTREPRISE ET DU SERVICE

3.1. Présentation de l'entreprise et du service

L'**objectif** est de favoriser la **socialisation des chiens** et de lutter contre l'**isolement des propriétaires** via des outils numériques favorisant les rencontres locales.

Les principaux axes de développement de WoofWalks sont :

- la création d'un **réseau communautaire localisé** pour les propriétaires de chiens
- la création d'un référencement de randonnées accessibles pour les chiens
- la création d'un référencement des parcs adaptés à la promenade de chien
- le développement d'une **application mobile** à partir du site web initial

Le projet évolue dans un environnement concurrentiel encore peu structuré. Les concurrents indirects incluent des groupes Facebook locaux ou des applications de rencontre généralistes, mais très peu de plateformes sont dédiées **exclusivement à la promenade collective de chiens**.

3.2. Objectifs du projet

Le projet web représente **le cœur de l'offre de WoofWalks**. Il s'agit d'une **application web** qui permet à la fois :

- de **publier des annonces** pour organiser ou référencer des promenades, randonnées ou parcs adaptés aux chiens
- de **gérer les participants**, les échanges, et les feedbacks associés à chaque événement.

Objectifs principaux :

- Mettre en relation des **propriétaires de chiens géographiquement proches**
- Permettre à chaque utilisateur de **créer une** balade ou d'en rejoindre une existante
- Offrir une **expérience communautaire** autour de la promenade canine, avec un **chat**, associé à chaque annonce, des **feedbacks illustrés**, et la **possibilité de commander un album photo souvenir**
- Gérer un **système de notifications paramétrables** (email, SMS) selon les préférences de chaque utilisateur
- Fournir un **outil d'administration** (back-office) pour modérer, analyser et enrichir les services

Il s'agit donc d'une **application en ligne** avec des logiques collaboratives, sociales et événementielles.

3.3. Cible adressée par le projet

Les utilisateurs visés partagent les caractéristiques suivantes :

- **Possèdent un chien,**
- **Souhaitent le socialiser** avec d'autres chiens
- Sont **motivés à sortir**, mais aimeraient le faire dans un cadre plus collectif
- Recherchent des **liens sociaux** et via leur passion pour les chiens
- Recherchent des échanges en termes de techniques d'éducation et des retours sur expériences

Trois segments cibles se dégagent :

1. **Jeunes actifs urbains (25-40 ans)**

Segment prioritaire

Connectés, souvent seuls ou en couple, souvent en appartement, et sensibles aux services communautaires numériques.

2. **Familles suburbaines ou rurales**

Segment secondaire stratégique

Avec jardin, plusieurs animaux parfois, et un intérêt pour les sorties nature en groupe les week-ends.

3. **Seniors propriétaires de chiens**

Segment complémentaire

Très fidèles une fois engagés, intéressés par la dimension sociale et rassurante de la communauté, parfois orientés vers des promenades douces ou des parcs de proximité.

4. GESTION DE PROJET

4.1. Intervenants sur le projet

Moi-même.

4.2. Méthodologie

Je me suis basé sur les méthodes Agile, idéales pour un projet évolutif nécessitant une forte réactivité. La méthode **Scrum** en particulier, qui privilégie le travail par **itérations** courtes, qui permettent de développer, tester et livrer régulièrement des fonctionnalités tout en restant flexible face aux changements.

Bien que je sois l'unique intervenant sur ce projet, j'ai simulé des **sprints** en reprenant les **user stories**, que j'ai ensuite priorisées selon leur importance et évaluées en termes de difficulté grâce à la technique du **planning poker**. Cette démarche m'a permis de structurer mon travail, d'anticiper les efforts à fournir et d'adapter les priorités en fonction des besoins.

Pour organiser ces sprints, j'ai créé des tableaux **Trello** dédiés, facilitant le suivi des tâches, la visualisation de l'avancement, et le maintien d'un focus constant sur les objectifs à court terme.

Cette approche présente également un véritable avantage pour le client : elle favorise une **collaboration étroite et continue**, en permettant des retours fréquents sur les avancées du projet. Cela offre au client une meilleure visibilité, l'assurance que ses besoins sont bien compris et pris en compte, et surtout la possibilité d'**ajuster les priorités** ou de faire évoluer certaines fonctionnalités en cours de développement. Cette transparence contribue à instaurer une relation de confiance, tout en garantissant que le produit final soit en adéquation avec les attentes réelles du client.

4.3. Outils, planning et suivi

La gestion de projet s'est déroulée en plusieurs phases définies, allant de la planification initiale à l'évaluation des sprints terminés. J'ai formalisé les besoins fonctionnels à travers des **user stories**.

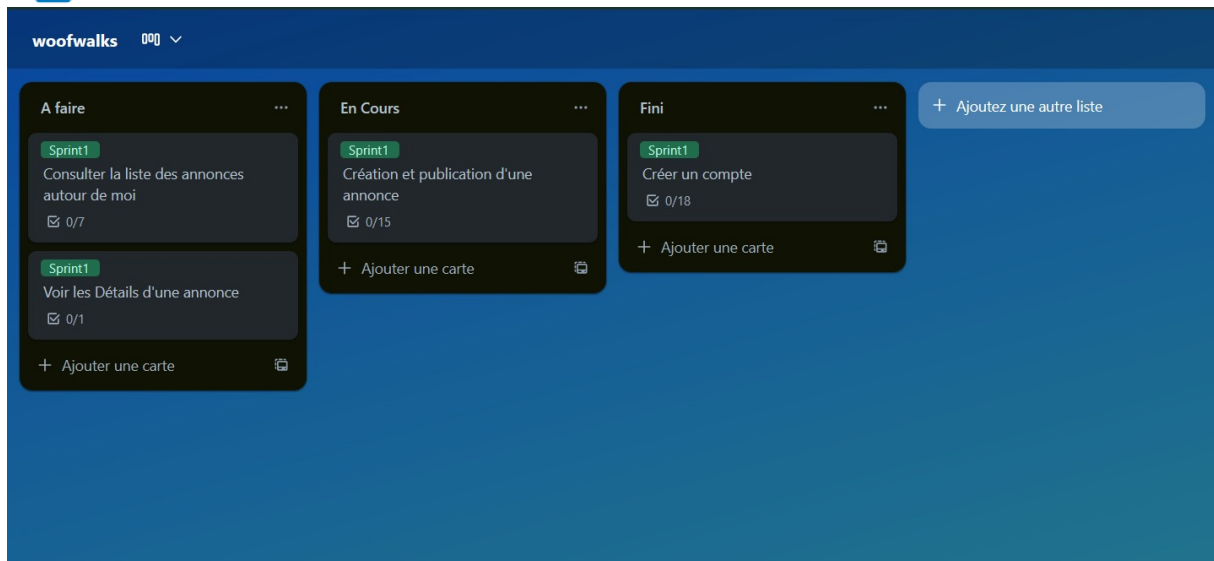
Pour prioriser efficacement ces fonctionnalités, j'ai appliqué la méthode **MoSCoW**, ce qui a orienté le planning et la répartition des efforts.

Le suivi des tâches et la planification des sprints ont été réalisés sur **Trello** qui utilise la méthode **Kanban** : chaque fonctionnalité était détaillée sous forme de cartes dans les tableaux de sprint, ce qui rendait facile la gestion visuelle et l'avancement précis du projet.

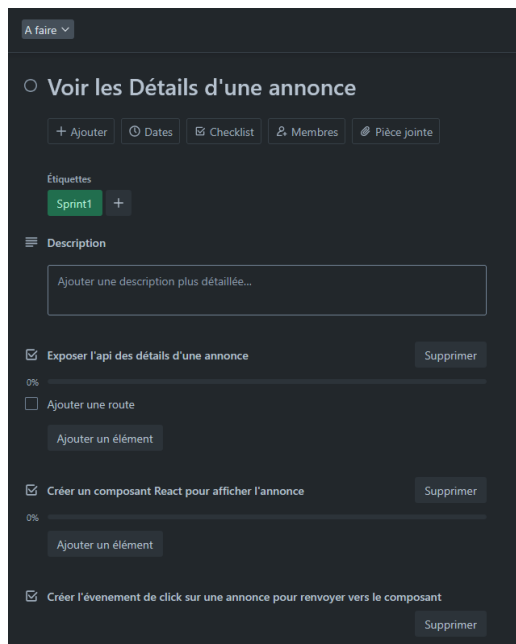
Enfin, lors des sessions de planification, j'ai de nouveau utilisé le planning poker pour estimer la charge de travail, ce qui m'a aidé à mieux gérer le temps et à anticiper les coûts liés au développement.

Cette approche agile et structurée m'a permis d'assurer un pilotage clair, de rester adaptable face aux imprévus, et de progresser efficacement vers un produit final conforme aux attentes.

Sprint 1 :



Détails d'une fonctionnalité à développer :



Coefficients estimations charge travail :

Coefficients planing poker	Durée
1	0,5
2	1
3	1,5
4	3
5	5

User Stories :

En bref qu...	Le user...	Afin de...	Points	Points Réels 15	Commentaire	Durée estimée
L'utilisateur	Choix mes préférences de notifications	Recevoir uniquement les informations qui m'intéressent sans être saturé	C		2) Mise en place d'un formulaire + configuration du système de notifications de l'utilisateur	1,5
Visiteur et l'utilisateur	Corriger les informations de l'inscription	Continuer avec l'inscription, pour toute information	C		2) Mise en place d'un formulaire de contact	1
L'utilisateur	S'inscrire en utilisateur ou du contenu non approuvé au contraire au RPO	Exclure un compte afin de garder un espace d'ui	C		2) Mise en place d'un formulaire de contact	1
L'utilisateur	Passer une annonce qui est une copie d'une annonce	Capter du temps si l'onglet sur les autres annonces	C		2) Mise en place d'un formulaire simple	1
L'utilisateur	Créer une annonce dans un lien référent comme par	Proposer un lien correct et adapté aux clients	C		0) Fil dans la partie Création d'une annonce	0
L'utilisateur	Corriger le profil des participants et leurs clients	Créer la compatibilité entre les annonces, voir si cela ne correspond	C		4) Travail en Front ++	3
L'utilisateur	Créer les notes des feedbacks des événements liés de l'organisation	Créer la qualité des événements passés et savoir si cela correspond à mes attentes	C		4) Travail en Front ++	3
L'utilisateur	Voir qui est inscrit à la table	Afin d'être une liste des participants et de l'interface privée	C		3	1,5
L'utilisateur	Avoir une vue sur une carte geo des tables privées	Voir les renseignements sur les tables privées	C		4) Mise en place d'une carte web et d'un système de tags selon la création des annonces et RPO	3
L'utilisateur	Créer un abonnement	Sélectionner les points qui ne plaident le plus pour leur marque	C		5) Travail Front ++ Recherche complexe si possible, UX facile	5
L'utilisateur	Voir tous les services autour de moi	Voir les renseignements de la table privée susceptible de me correspondre	C		Pour selon l'extension actuelle de l'utilisateur (possibilité à tout moment de voir dans « Cher et gérer un compte ») - Nécessaire l'implémentation du service de géolocalisation 3) Travail en Front ++	1,5
Système	Voir l'achat de mon abonnement	Recevoir l'abonnement que j'ai acheté	C		5) Validation par email, sécurisé, gestion erreurs, envoi confirmation, envoi paiement confirmé au RPO.	5
L'utilisateur inscrit à une table	Recevoir des notifications de tout changement de services	Etre immédiatement au courant de l'achat de lien ou d'autres détails éventuels	C		4) - Nécessaire la mise en place d'un système de notifications	3
L'utilisateur inscrit à une table	Recevoir des notifications du chat	Recevoir des échanges entre participants	C		4) - Nécessaire la mise en place d'un système de notifications	3
L'utilisateur inscrit à une table	Recevoir des notifications lorsqu'une place dans la table se libère	Pouvoir m'inscrire sans payer d'autres ne permettant la place	C		4) - Nécessaire la mise en place d'un système de notifications	3
L'utilisateur inscrit à une table	Envoyer des feedbacks commentaires des points du site et une note globale	Partager mon expérience et améliorer la qualité des points	C		4) Mise en place d'un formulaire avec notes et points	3
L'utilisateur inscrit à une table	Créer les tickets liés du table à l'utilisateur invité	Voir les services des autres participants et savoir l'événement à tous les points et commentaires	C		2	1
Visiteur	M'inscrire	Accéder aux services de l'application et accéder aux tables	M		5) Création de compte avec pseudo, mot de passe, photo, vérification et confirmation par email (ou autre méthode)	5
L'utilisateur	Mes commentaires	Voir les tables des commentaires de l'application	M		2) Mise en place d'un formulaire de contact	1,5
L'utilisateur	Mes commentaires	Quitter l'application	M		2	1
L'utilisateur	M'inscrire à une table	Participer à une table qui ne correspond	M		2) Script click sur bouton + Email confirmation par email	1
L'utilisateur	Créer toutes les annonces publiques	Se connecter avec l'utilisateur	M		1	0,5
L'utilisateur	Recevoir des notifications des utilisateurs lorsqu'un contenu est approuvé	Voir le contenu approuvé	M		2	1
L'utilisateur	Accéder et modifier les profils des utilisateurs	Modifier les comptes en cas de demande explicite d'un utilisateur ou d'un utilisateur si besoin	M		3	1,5
L'utilisateur	Suivre un contenu utilisateur	Repondre à la demande d'un utilisateur ou d'un utilisateur qui a répondu par les CCI	M		2	1
Modérateur	Modifier les ratings dans les CCI des tables	Voir le contenu approuvé	M		2	1
Modérateur	Modifier les ratings dans les CCI des tables	Voir le contenu approuvé	M		3	1

4.4. Objectifs de qualité

Voici les qualités indispensables d'un site web. :

(source <https://burguindigital.com/qualite-site-web/>)

- **Un design attractif et professionnel** – Donne une bonne première impression et renforce la crédibilité
- **Un site qui se charge rapidement** – Améliore l'expérience utilisateur et réduit le taux de rebond
- **Une ergonomie fluide et intuitive** – Facilite la navigation et retient l'attention des visiteurs
- **Une organisation claire** – Permet de trouver rapidement l'information recherchée
- **Du contenu de qualité** – Renforce la confiance et favorise le référencement naturel (SEO)
- **Une sécurité à toute épreuve** – Protège les données des utilisateurs et inspire confiance
- **Des éléments de réassurance** – Témoignages, garanties, labels... pour rassurer et convertir

5. SPÉCIFICATIONS FONCTIONNELLES

5.1. Contraintes du projet et livrables attendus

5.1.1. Criticité de l'application

L'application WoofWalks vise à organiser des balades et événements pour propriétaires de chiens. Son bon fonctionnement conditionne la qualité de l'expérience utilisateur, mais une interruption temporaire ne génère pas de perte financière directe. Cependant, une indisponibilité prolongée impacte la satisfaction des utilisateurs réguliers et peut nuire à l'image de la plateforme.

La population utilisatrice est composée :

- D'utilisateurs non-inscrits consultant les événements publics.
- D'utilisateurs inscrits (propriétaires) créant ou rejoignant des événements.
- De modérateurs ou administrateurs gérant les contenus.

Nombre estimé d'utilisateurs au lancement :

- 100–200 utilisateurs inscrits.
- 50 utilisateurs actifs mensuels.

Disponibilité visée :

- 99 % (hors opérations de maintenance planifiées).
- Support assuré 5j/7 via formulaire de contact.

Indisponibilité temporaire :

- Impact limité à la gêne utilisateur (annulations impossibles, accès aux informations restreint).
- Aucune incidence financière directe mais perte potentielle d'opportunités d'inscription ou d'engagement.

5.1.2. Services tiers

L'application WoofWalks fait appel à plusieurs services tiers pour enrichir l'expérience utilisateur et faciliter certaines fonctionnalités :

- **Leaflet** : bibliothèque JavaScript utilisée pour l'affichage des cartes interactives sur les pages événements.
- **Fournisseur météo (à intégrer en V2)** : une API météo sera intégrée pour afficher la météo prévue le jour de l'événement.
- **Leaflet** : cartes interactives pour la géolocalisation des balades.

5.1.3. Livrables attendus

- Cahier des charges
- Spécifications fonctionnelles & techniques
- Maquettage & design
- Développement
- Intégration
- Gestion de base de données ou autres actifs existants

5.2. Architecture logicielle du projet

Le projet WoofWalks est développé selon une architecture **Multicouche**, utilisant Symfony et React. L'ensemble est conteneurisé sur Docker.

Frontend : application web réalisée en **React**, qui gère l'interface utilisateur, les interactions, l'affichage des cartes (via Leaflet), et la consommation des API REST du backend.

Backend : API REST développée avec **Symfony** et **API Platform**. API Platform permet de construire rapidement une API, avec des fonctionnalités intégrées comme le routage, la pagination, la validation et la sérialisation des données. Cette solution facilite la création, la lecture, la mise à jour et la suppression des entités.

Toutefois, certaines fonctionnalités complexes nécessitent un traitement spécifique. En effet, les formulaires soumis depuis le Frontend React peuvent contenir un mélange de types de données, incluant à la fois des données textuelles (titre, description, date) et des fichiers (photos à uploader). Or, API Platform gère principalement des requêtes au format JSON et ne supporte pas nativement l'upload de fichiers dans ses opérations standard.

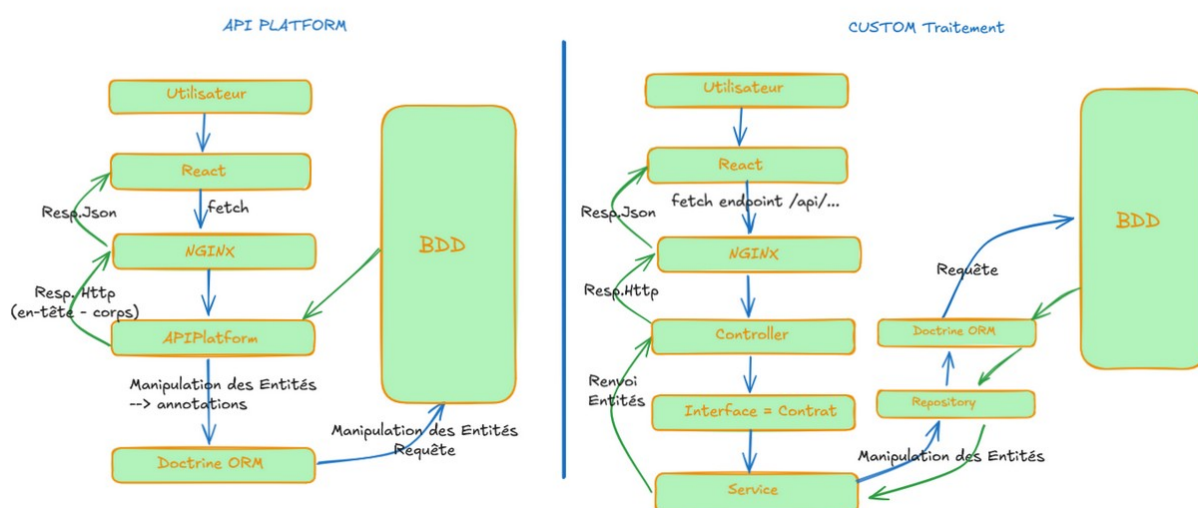
C'est pourquoi des **contrôleurs personnalisés** ont été développés pour traiter ces formulaires, permettant de gérer simultanément les fichiers et les autres champs de l'annonce.

Base de données : un système de gestion de base de données relationnelle **MySQL** est utilisé pour stocker les données de l'application.

Le code source est géré avec Git, assurant le versioning et le suivi des évolutions.

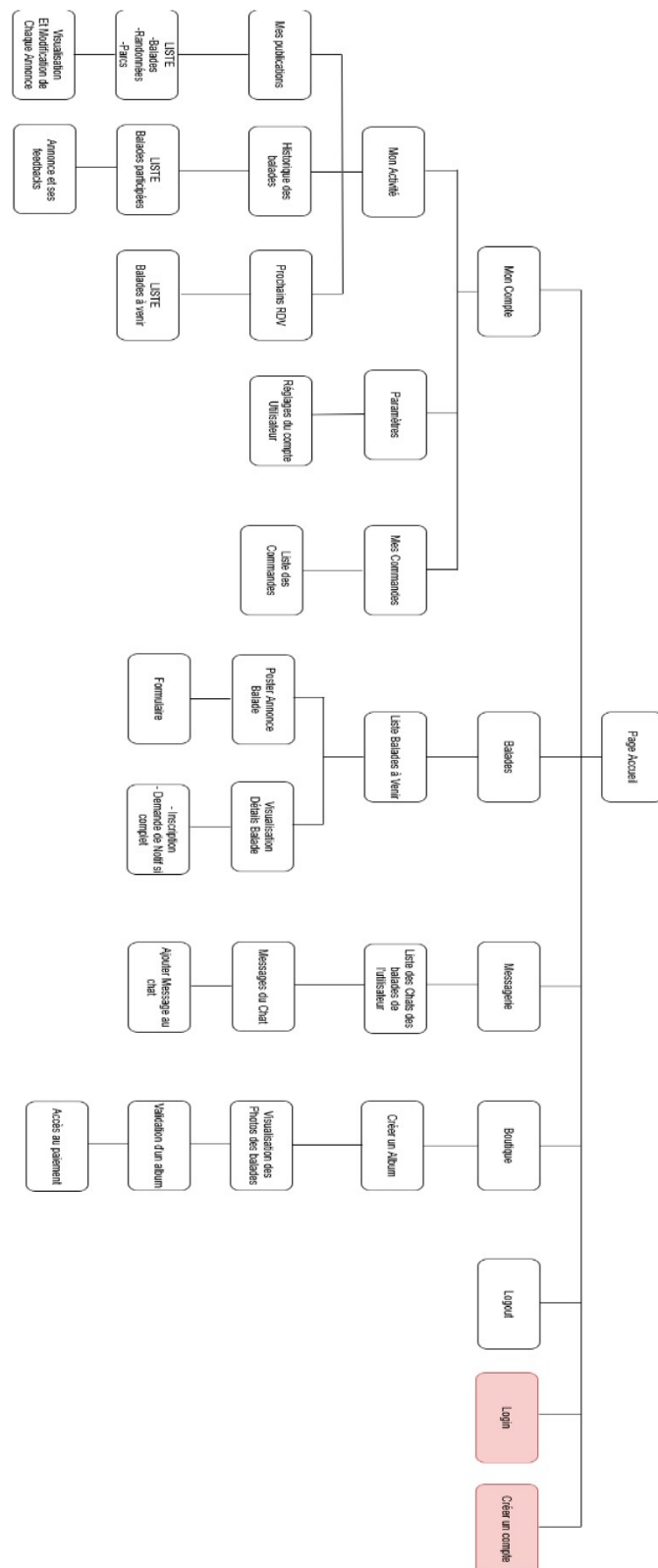
L'architecture est **multicouche** puisque Symfony repose basiquement sur le mode **MVC**. J'ai intégré des **interfaces et des services** (couche **métier**) auquel mes **contrôleurs** (couche **présentation**), sur requête de React, font appels. La couche métier consomme les **Repositories** pour **accéder aux données**.

Approche hybride : Utilisation d'api platform et traitement personnalisé de la logique métier plus complexe .



5.3. Maquettes et enchaînement des maquettes

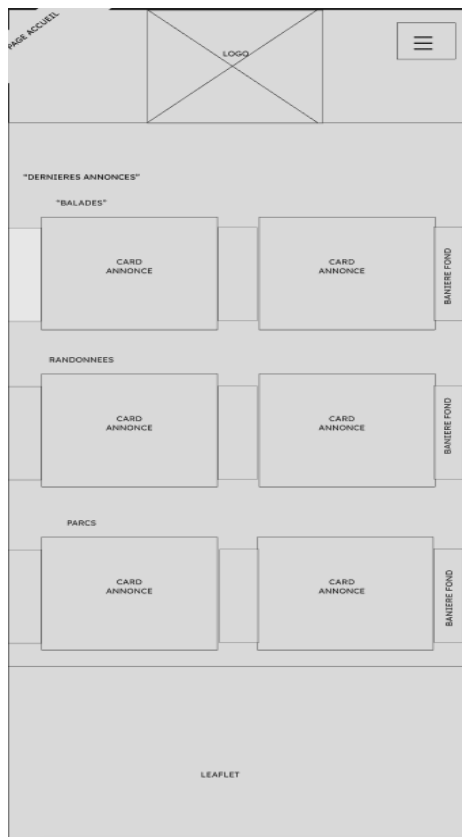
5.3.1. Cartographie



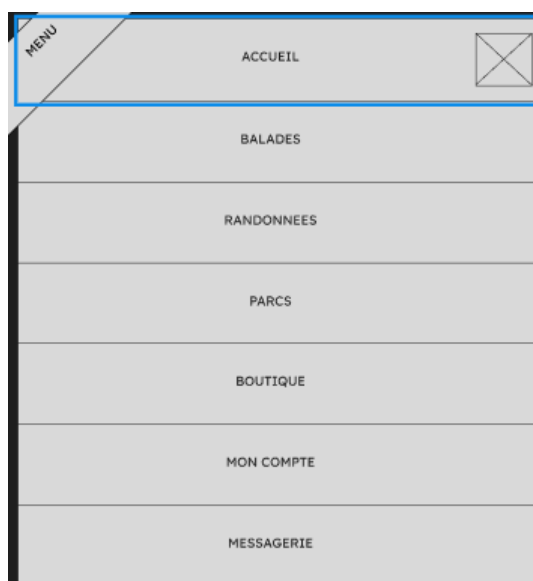
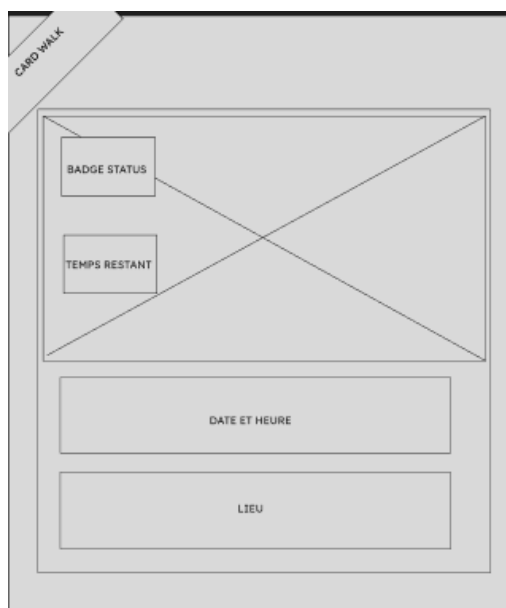
5.3.2. Maquettes

Wireframes :

Page d'accueil et Page Liste des balades :



Détails d'une balade et menu de navigation :



Post d'une balade, détails d'une balade et page Chat :

UNE BALADE

LOGO

BALADES REGULIERES

CARD ANNOUNCE

CARD ANNOUNCE

BOUTON FOND

Titre

Date/heure du rdv

Nombre Max Participants

Type de Lieu

Point de RDV

Thème special

Rdv régulier ?

Image

Description

TEXTE

PUBLIER

OU

REPUBLIER

FOOTER

DETAILS BAL

LOGO

waik_details

TITRE DE L'ANNONCE

NB PARTICIPANTS ICON METEO

ORGANISATEUR

DESCRIPTION

VILLE

POINT DE RDV

THEME SPECIAL

BTN COMPLET

RECEVOIR ALERTE

OU

PARTICIPER

Chat

LOGO

CARD ANNOUNCE

CARD ANNOUNCE

BOUTON FOND

utilisateur

date - heure

Message

utilisateur

date - heure

Message

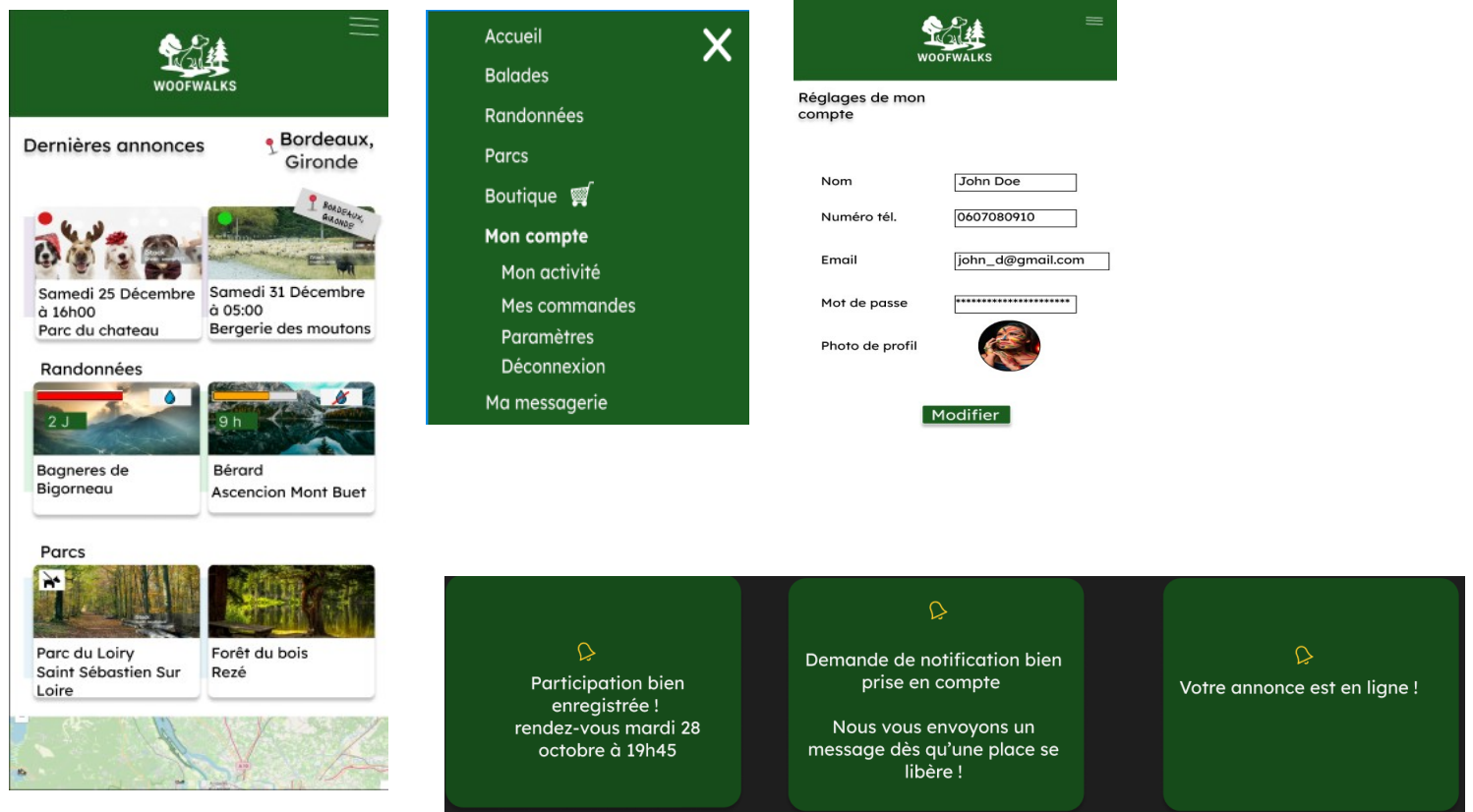
utilisateur

date - heure

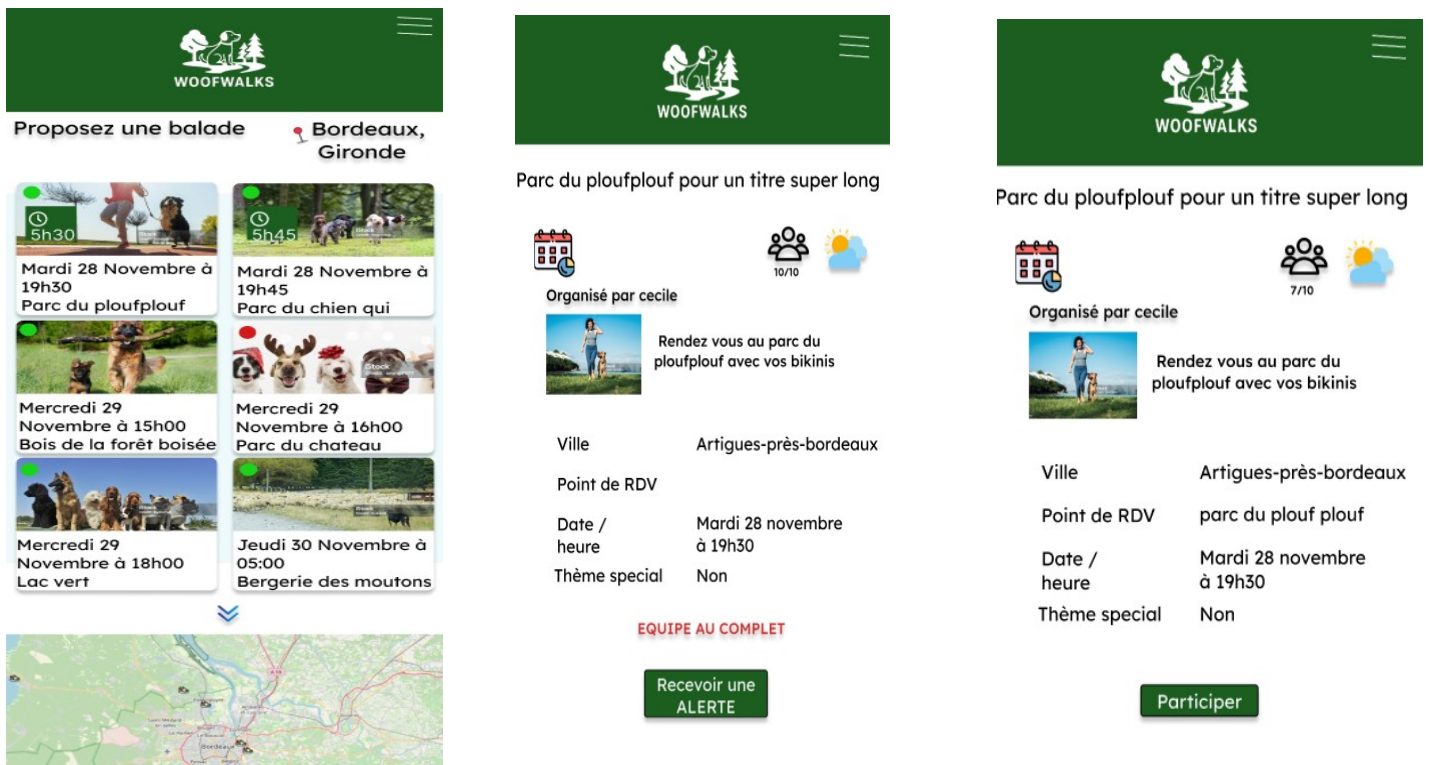
Message

Maquettes :

Page Accueil, Menu Navigation, Administration Compte Utilisateur, Popups :



Page Liste des balades et détails d'une balade (disponible et non disponible) :



Post et Re-post d'une annonce :

Parc du ploufplouf pour un titre super long

Organisé par cecile

7/10

Rendez vous au parc du ploufplouf avec vos bikinis

Ville Artigues-près-bordeaux

Point de RDV parc du plouf plouf

Date / heure Mardi 28 novembre à 19h30

Thème special Non

Republier

Vos balades régulières

Mardi 28 Novembre à 19h30
Parc du ploufplouf

Mardi 28 Novembre à 19h45
Parc du chien qui sent

Ajouter une balade

Titre

Date/heure du rdv

Nombre Max Participants

Type de Lieu ☐ Personnalisé ☐ Parc

Point de RDV

Thème special

Rdv regulier ? ☐

Image

Description

Votre message...

Publier

Vos balades regulieres

Mardi 28 Novembre à 19h30
Parc du ploufplouf

Mardi 28 Novembre à 19h45
Parc du chien qui sent

Republiez une balade

Titre **Rendez vous au parc du ...**

Date/heure du rdv **28/11/2024 19:30**

Nombre Max Participants **7**

Type de Lieu ☐ Personnalisé ☒ Parc

Point de RDV **Parc du ploufplouf pour un...**

Thème special **Bouviens Bernois**

Rdv regulier ? ☒

Image

Description

Hello tout le monde, je vous propose un rendez-vous à artigues avec vos bouviens en priorité. Chiens de plage autorisés mais priorité aux chiens de montagne quand même.
Non sans blaguer, l'idée est de rencontrer quelques autres maîtres de bouviens mais on sera heureux de les voir s'amuser avec n'importe quel copain bien sûr.
A tout bientôt

Publier

Chat et Commandes d'albums photos :

Bordeaux Gironde

CHATS

Mardi 28 Novembre à 19h30
Parc du ploufplouf

Mardi 28 Novembre à 19h45
Parc du chien qui sent

06:22
Salut les gens, vous avez de l'eau pour ce soir ?
Parce que moi oui mais j'ai envie de parler parce que j'attends le bus.

07:40
Non mais j'ai pas envie de parler alors je te réponds !

07:42
Ok, j'ai du coca pour nous et du kirsh pour les chiens donc pas de soucis. Je suis à la natation et il y a du monde dans l'eau alors je suis aux aguets je ne peux pas trop vous parler vous voyez ?

07:42
Ha c'est bien ça d'écrire sous l'eau, tu pourrais peut-être te spécialiser dans la rédaction d'articles sous-marins mais qui ne parlent pas de sous-marin du tout?

17:15
Bon alors on se voit quand même, merci pour le coca ...

Et du coup, ok pour 19 heures ??

Envoyer

Créez votre album photo

Un album unique pour votre compagnon unique

Créer mon album maintenant

Impression professionnelle
100 % personnalisable

Mes Commandes

Commande en cours

**** Les balades de Luna ****
Commande #CMD-20250706-001
Commandé le 01 juillet 2025
Statut : En production
Livraison estimée : 15 juillet 2025

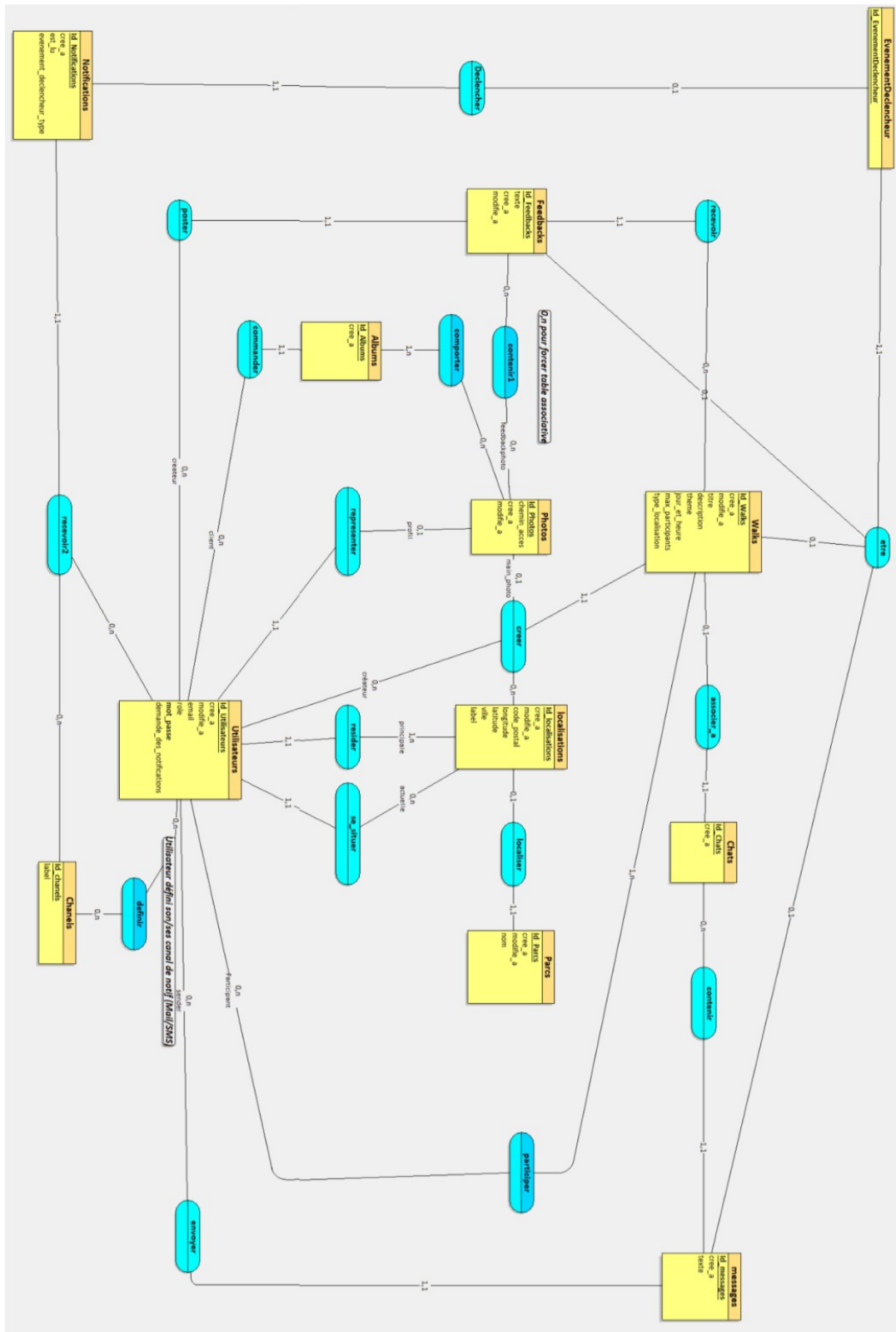
**** Les balades de Lilo ****
Commande #CMD-20250706-002
Commandé le 01 juillet 2025
Statut : En production
Livraison estimée : 15 juillet 2025

Historique des commandes

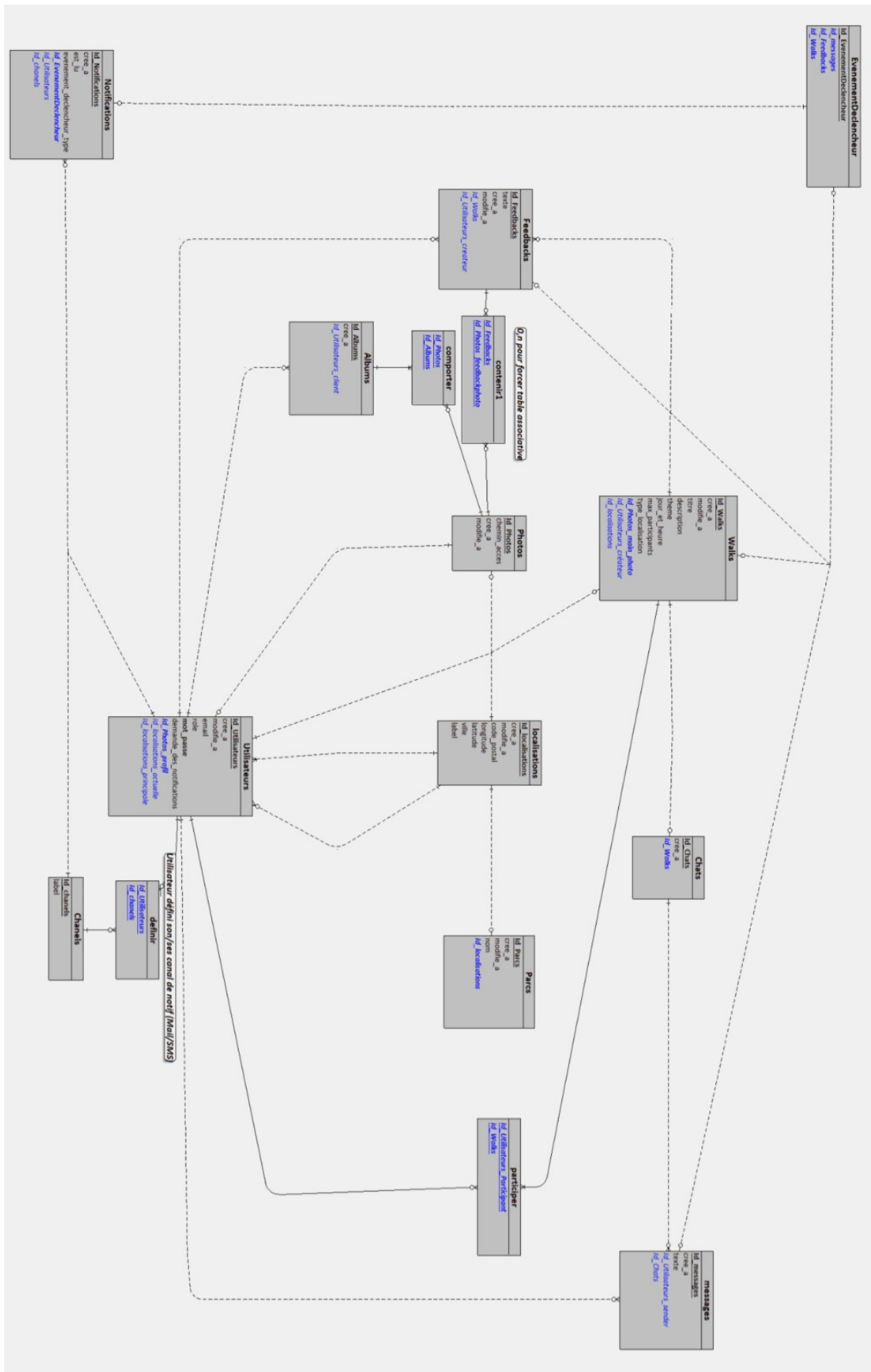
**** Les balades de Yollanda ****
Commande #CMD-20241012-001
Commandé le 10 octobre 2024
Statut : En production
Livraison estimée : 15 octobre 2024

5.4. Modèle entités-associations et modèle physique de la base de données

5.4.1. MCD (MERISE)



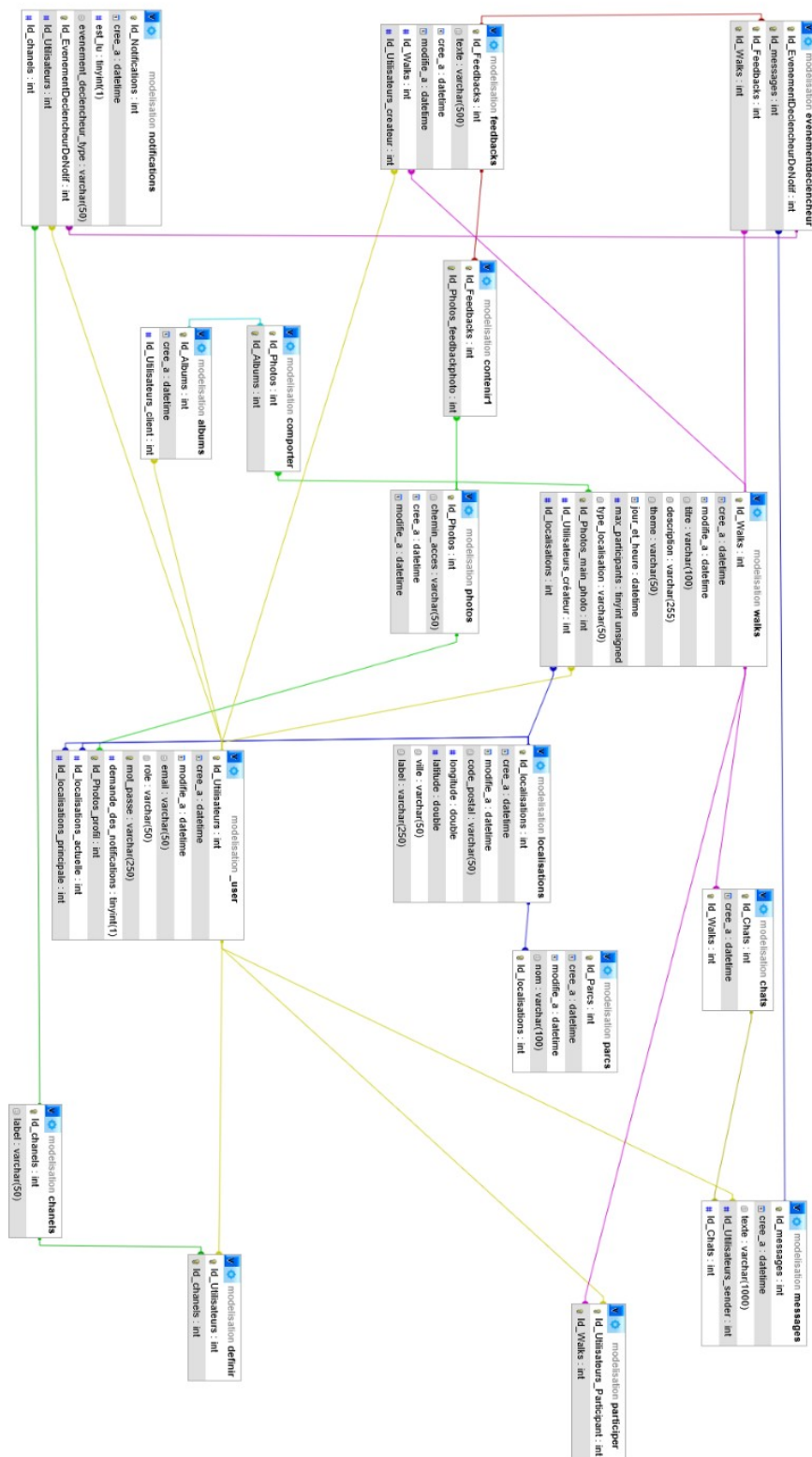
5.4.2. MLD (MERISE)



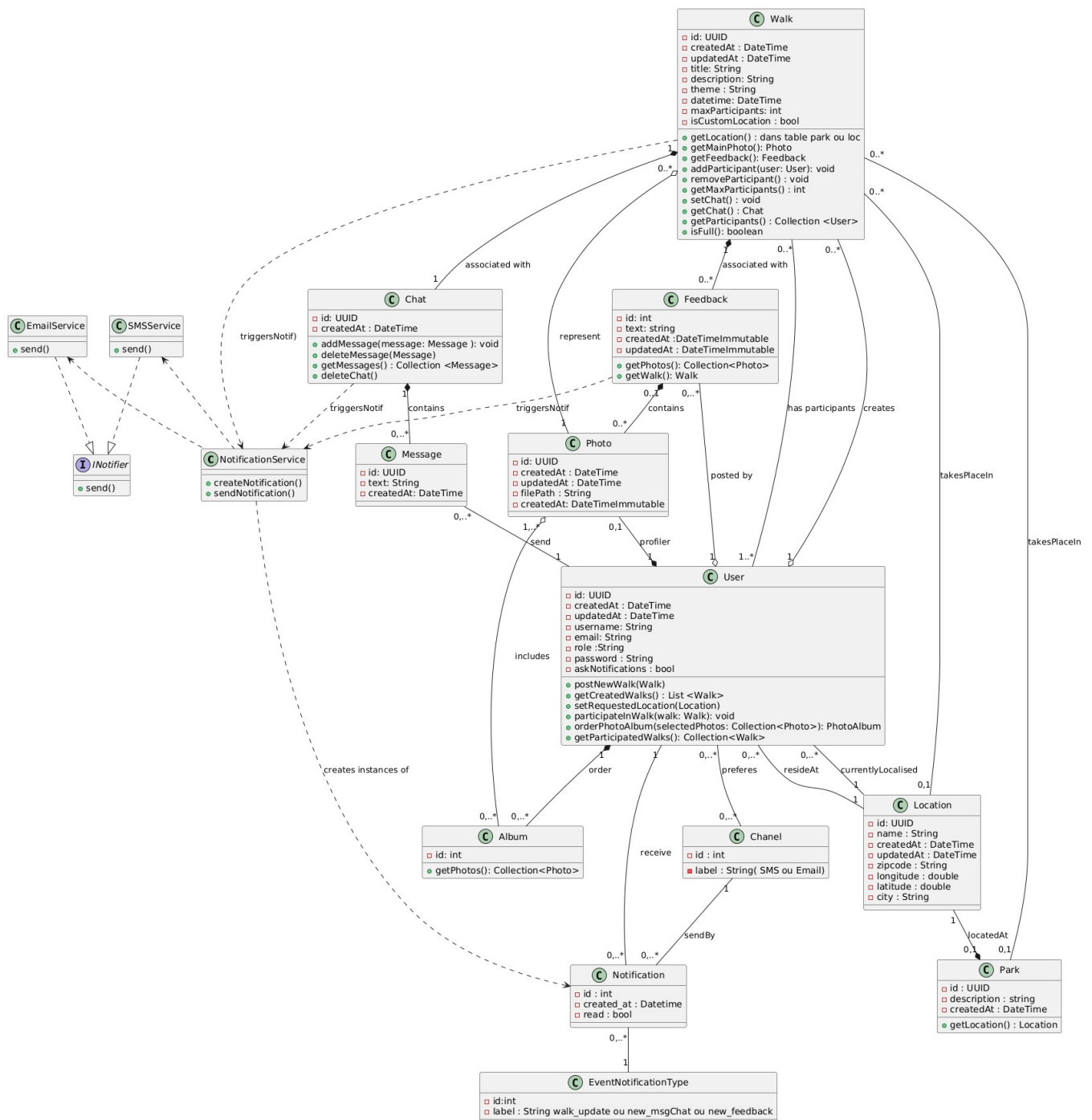
5.4.3. Dictionnaire de données

Code Mnémonique	Type	Désignation	Taille	Remarques
User				
id_utilisateur	N	Identifiant unique de l'utilisateur	5	
cree_a	DATE	Heure de la création		
nom_utilisateur	A	Nom de l'utilisateur	100	
rôle	A	Rôle	50	
demande_notifications	B	Demande des notifications		
email	A	Adresse email de l'utilisateur	150	
mot_de_passe	A	Mot passe de l'utilisateur	250	
Walk				
id_walk	N	Identifiant unique de la promenade	5	
cree_a	DATE	Date de création de la promenade		
modifie_a	DATE	Heure de la modification		
date_et_heure_balade	DATE	Heure de la promenade		
titre	N	Titre de la promenade	2	
description	A	Contenu de la promenade	500	
thème	A	Thème spécifié	100	Peut être NULL
max_participants	N	Nombre maximum de participants	3	Parc ou custom pour chercher dans la bonne table
type_localisation	A	Type de localisation		
Feedback				
id_feedback	N	Identifiant unique du Feedback	5	
cree_a	DATE	Date de création du Feedback		
texte_feedback	A	Contenu du Feedback	500	
Photo				
id_photo	N	Identifiant unique de la photo	5	
chemin_fichier	AN	Chemin de la photo	255	
Album				
id_album	N	Identifiant unique de l'album	5	
cree_a	DATE	Date de création de l'album		
Chat				
id_chat	N	Identifiant unique du chat	5	
cree_a	DATE	Date de création du Chat		
ChatMessage				
id_message	N	Identifiant unique du message	5	
cree_a	DATE	Date de création du message		
texte	A	Contenu du message	255	
Album				
cree_a	DATE	Date de création de l'album		
Evenement_declencheur				
id_evenement_declencheur	N	Identifiant unique de l'événement déclencheur de notification	5	Exemple : nouveau message chat Ou annonce modifiée...
Notification				
id_notification	N	Identifiant unique de la notification	5	
cree_a	DATE	Date de création		
est_lu	B	La notification est lue	50	
Chanel				
id_channels	N	Identifiant unique du chanel	5	
label	A	Libellé du chanel	50	
Parc				
id_parc	N	Identifiant unique du parc	5	
label	A	Libellé du parc	50	
cree_a	DATE	Date de création du Parc		
modifie_a	DATE	Heure de la modification		
Localisation				
id_localisation	N	Identifiant unique du chanel	5	
label	A	Libellé de la localisation	50	
cree_a	DATE	Date de création de la localisation		
modifie_a	DATE	Heure de la modification		
longitude	N	Longitude de la localisation	250	
latitude	N	Latitude de la localisation	250	
code_postal	AN	Code Postal de la localisation	50	
ville	AN	Ville de la localisation	250	

5.4.4. MPD (MERISE)



5.4.5. Diagramme de classes (UML)



5.5. Script de création et/ou de modification de la base de données

L'utilisation de Doctrine ORM avec Symfony simplifie la gestion de la base de données et évite les erreurs humaines liées à la rédaction manuelle des scripts SQL.

5.5.1. Script de création

Je n'ai pas rédigé manuellement de script SQL pour la création ou la modification de la base de données, car j'ai utilisé le framework **Symfony** associé à **Doctrine ORM**.

Doctrine permet de manipuler des bases de données en utilisant des objets au lieu d'écrire du SQL directement.

Dans ce cadre, la structure de la base est donc **générée automatiquement à partir des entités** que j'ai définies. Symfony permet de maintenir la base à jour en utilisant des **migrations Doctrine**, qui créent automatiquement les requêtes SQL à partir des modifications apportées au code.

La commande `symfony console make:entity` permet de créer une entité (représentant une table).

La commande `symfony console make:migration` génère le script SQL correspondant aux changements.

Puis `symfony console doctrine:migrations:migrate` applique les modifications à la base.

Exemple d'une requête SQL générée lors de la création d'une entité `walkAlertRequest` :

```
<?php
declare(strict_types=1);

namespace DoctrineMigrations;

use Doctrine\DBAL\Schema\Schema;
use Doctrine\Migrations\AbstractMigration;

/**
 * Auto-generated Migration: Please modify to your needs!
 */
final class Version20250614142832 extends AbstractMigration
{
    public function getDescription(): string
    {
        return '';
    }

    public function up(Schema $schema): void
    {
        // this up() migration is auto-generated, please modify it to your needs
        $this->addSql('CREATE TABLE walk_alert_request (id INT AUTO_INCREMENT NOT NULL, user_id INT DEFAULT NULL, walk_id INT DEFAULT NULL,
        $this->addSql('ALTER TABLE walk_alert_request ADD CONSTRAINT FK_629861CAA76ED395 FOREIGN KEY (user_id) REFERENCES `user` (id)');
        $this->addSql('ALTER TABLE walk_alert_request ADD CONSTRAINT FK_629861CA5EEE1B48 FOREIGN KEY (walk_id) REFERENCES walk (id)');
    }

    public function down(Schema $schema): void
    {
        // this down() migration is auto-generated, please modify it to your needs
        $this->addSql('ALTER TABLE walk_alert_request DROP FOREIGN KEY FK_629861CAA76ED395');
        $this->addSql('ALTER TABLE walk_alert_request DROP FOREIGN KEY FK_629861CA5EEE1B48');
        $this->addSql('DROP TABLE walk_alert_request');
    }
}
```

5.5.2. Script de modification

Lorsqu'une entité est modifiée (par exemple, ajout d'un champ ou modification d'une relation), la commande `symfony console make:migration` génère un nouveau script SQL qui inclut uniquement les changements nécessaires. Ce script est ensuite appliqué à la base via `doctrine:migrations:migrate` sans perdre les données existantes.

Exemple simplifié d'une migration :

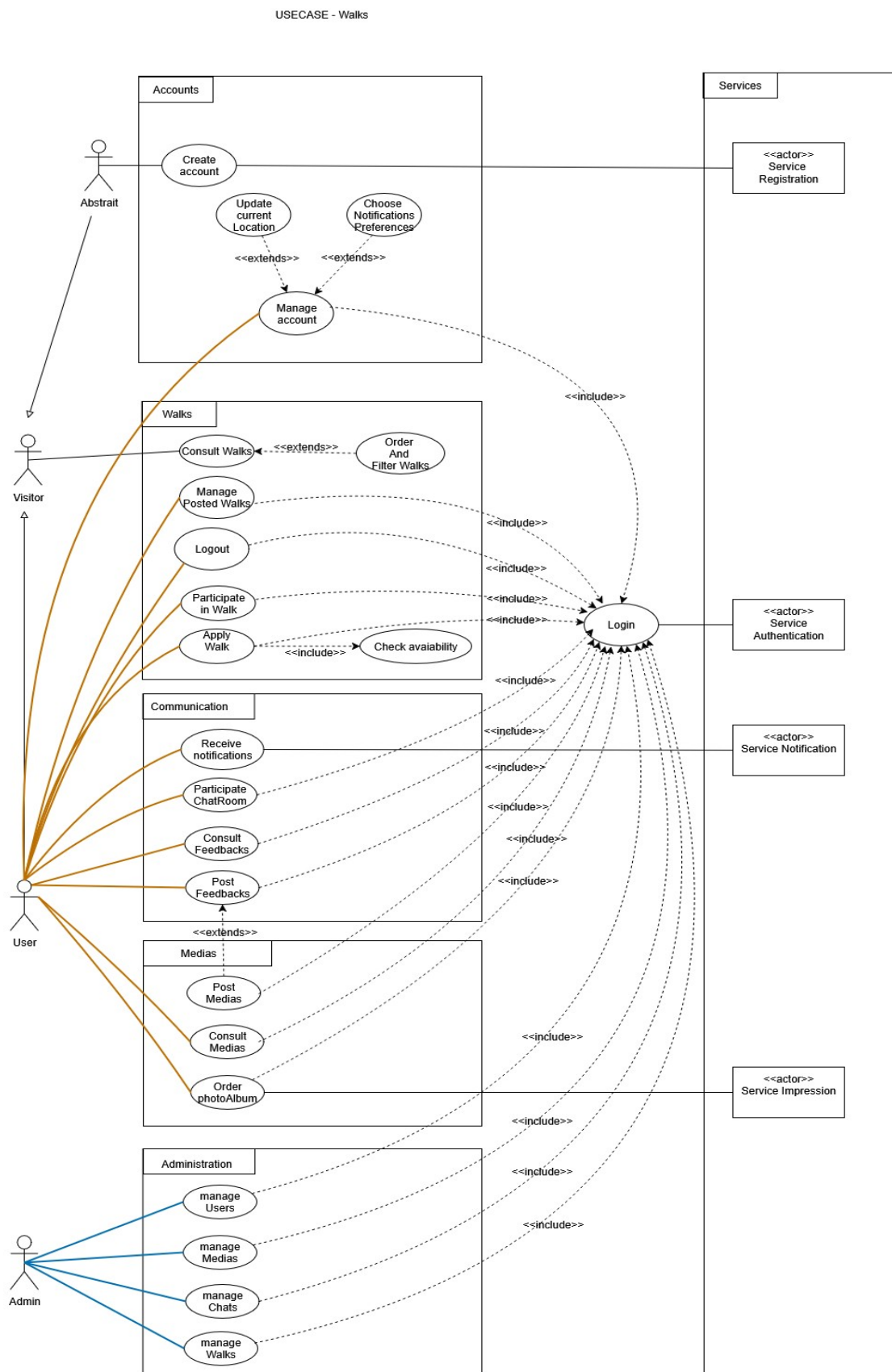
```
/**
 * Auto-generated Migration: Please modify to your needs!
 */
final class Version20250614170645 extends AbstractMigration
{
    public function getDescription(): string
    {
        return '';
    }

    public function up(Schema $schema): void
    {
        // this up() migration is auto-generated, please modify it to your needs
        $this->addSql('ALTER TABLE notification ADD content VARCHAR(255) NOT NULL');
    }

    public function down(Schema $schema): void
    {
        // this down() migration is auto-generated, please modify it to your needs
        $this->addSql('ALTER TABLE notification DROP content');
    }
}
```

5.6. Diagramme du comportement des fonctionnalités de type cas d'utilisations

5.6.1. Diagramme de cas d'utilisation (UML)



5.7. Fonctionnalités détaillées les plus significatives

5.7.1. Fonctionnalité n° 1 : S'inscrire à une balade

Description :

L'utilisateur souhaite s'inscrire à une balade.

Scénario Nominal :

- L'utilisateur accède à la page de liste des walks
- Il clique sur une annonce de walk
- Le système vérifie qu'il est connecté
- Le système affiche la page de détails de la balade.
- L'utilisateur consulte les détails
- Il vérifie si la promenade est encore disponible
- Si oui, il clique pour rejoindre la promenade :
 - Le système mémorise la demande, puis tente de la sauvegarder en base de données
 - Si la sauvegarde réussit :
 - Le système affiche un message de validation
 - Le système cherche la liste des autres participants et les notifie qu'un participant s'ajoute à la balade
 - Le système cherche les informations relatives à l'affichage de la version participant du chat et les affiche
 - L'utilisateur est redirigé vers la page de la promenade (version participant)
- Si non, il peut cliquer pour demander une notification lorsqu'une place se libère
- Le système mémorise la demande, puis tente de la sauvegarder en base de données
- Si la sauvegarde réussit :
 - Le système affiche un message de validation

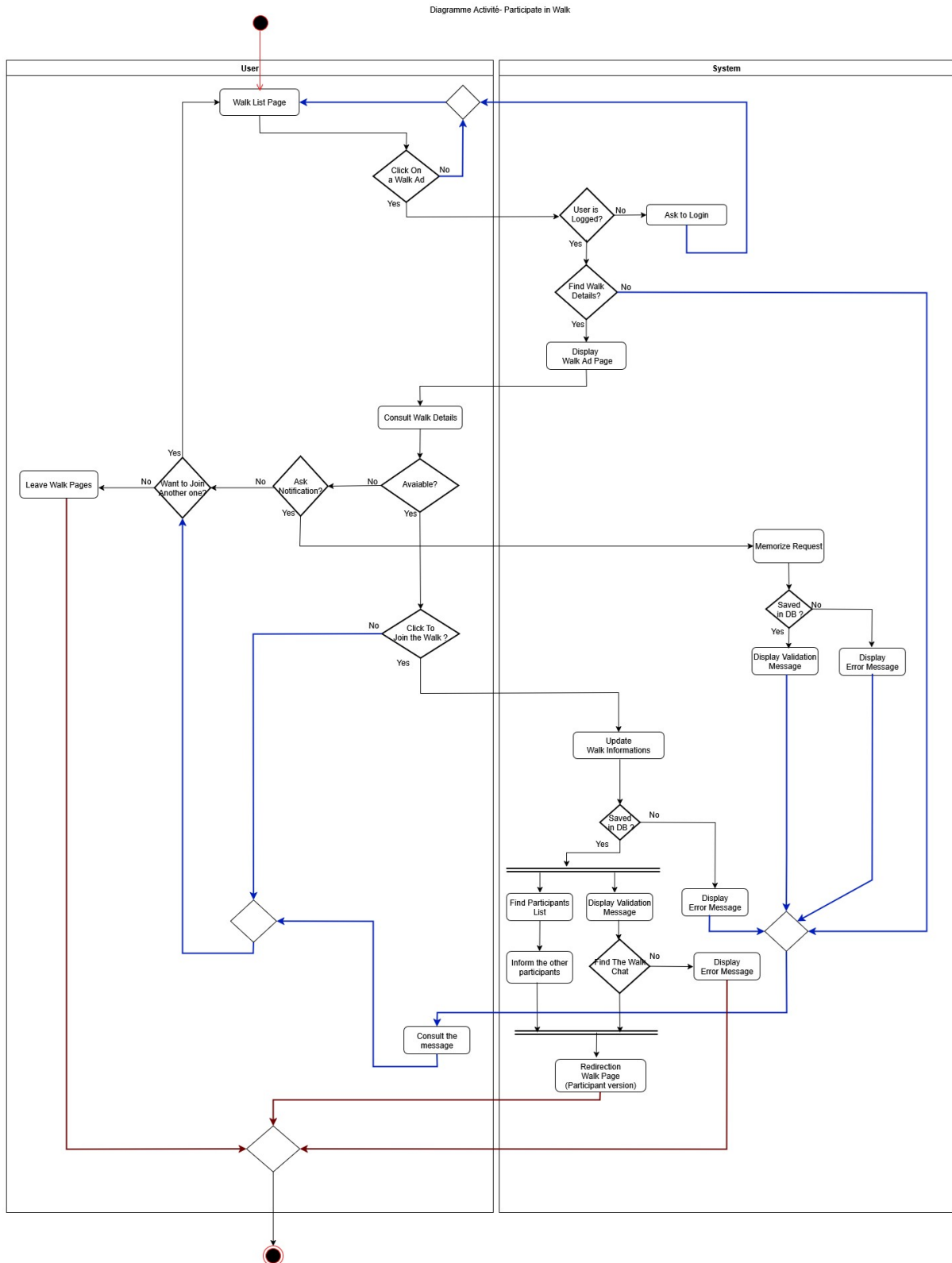
Enchaînements si Erreurs :

Voici les scénarios d'enchaînement si erreur :

- Si l'utilisateur n'est pas connecté :
 - L'utilisateur est invité à se connecter
- Les détails de l'annonce n'ont pas été trouvés en BDD
 - Le système affiche un message d'erreur
- Si l'utilisateur demande à recevoir une notification si une place se libère, mais que la procédure échoue
 - Le système affiche un message d'erreur
- Si l'utilisateur demande à rejoindre la balade, mais que la sauvegarde échoue :
 - Le système affiche un message d'erreur
- Si le système ne trouve pas les détails relatifs à l'affichage de la version participant de la balade
 - Le système affiche un message d'erreur

Diagramme d'activité

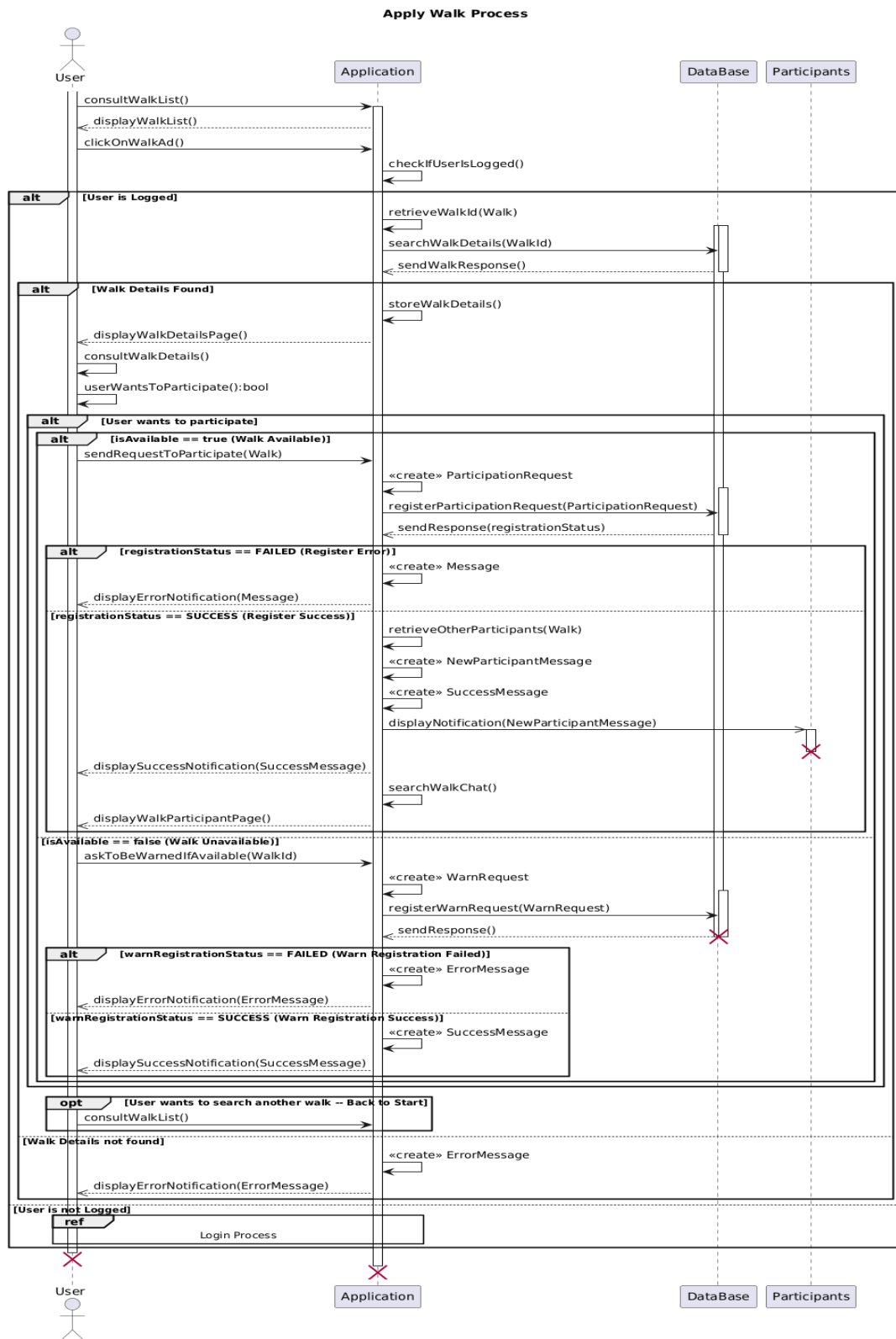
Ce diagramme illustre le processus qu'un utilisateur suit pour consulter et participer à une balade



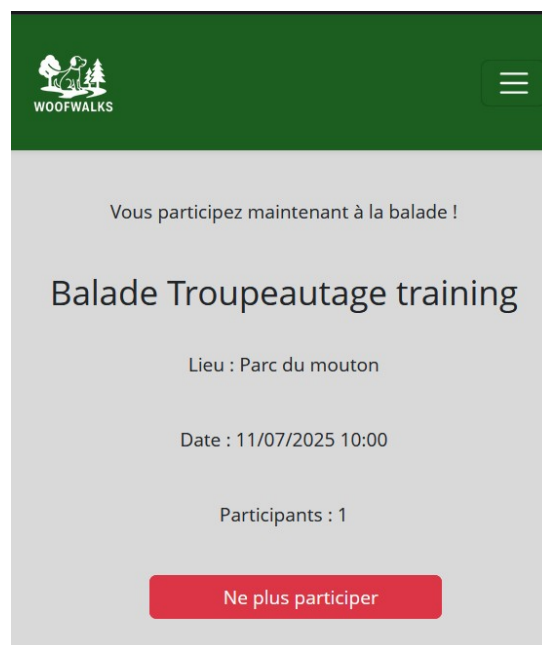
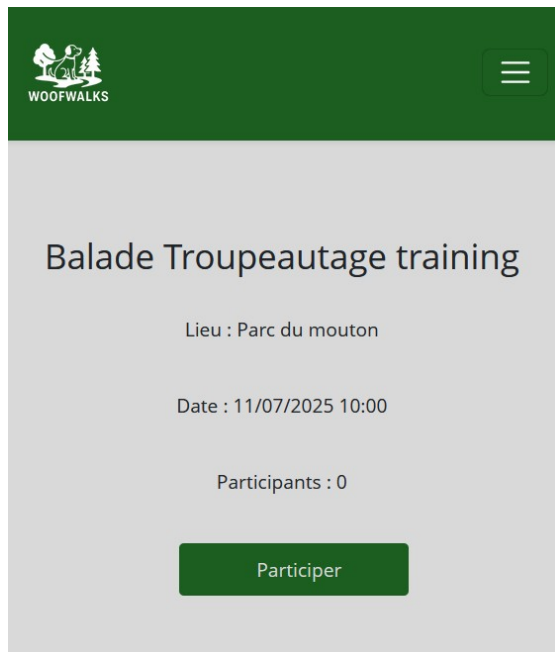
Données/Traitements

En entrée	Traitement	En sortie	Contrôles
Clic sur une annonce : - Id de la balade - Id de l'utilisateur	-Vérification que l'utilisateur est connecté -Récupération des détails de la balade à partir de son ID	Page avec les détails de la balade Ou Message Erreur	-L'utilisateur est connecté - La balade existe
Clic sur « rejoindre la balade » : - Id de la balade - Id de l'utilisateur	- Récupération de la balade - Récupération des données de l'utilisateur - -Enregistrement en BDD -Récupération des informations nécessaires à l'affichage de la page version Participants (Chat)	- Affichage Message de confirmation -Envoi de notification aux autres participants - Recherche les infos pour affichage de la version participant (Accès au Chat) - Redirection vers page version participant	La Balade existe et n'a pas atteint son nombre max de participants
Clic pour demander une Notification si une place se libère : - Id de la balade - Id de l'utilisateur	- Enregistrement de la demande	- Enregistrement en BDD et Message de confirmation de la demande de Notification	- L'utilisateur existe - La balade existe

Diagramme de séquence



Écran/Affichage



5.7.2. Fonctionnalité 2 : Poster une annonce pour une balade

Description

L'utilisateur poste une annonce pour planifier une balade.

Scénario Nominal :

- L'utilisateur Consulte la liste des balades
- Il clique sur « Poster une balade »
- Le système vérifie si l'utilisateur est connecté
- S'il est bien authentifié : le système lui renvoi le formulaire d'ajout d'une annonce
- L'utilisateur remplit les champs :
 - Champs standards (date et heure du RDV, nombre participants...)
 - Champ Localisation :
 - L'utilisateur choisi si le point de RDV est référencé comme parc ou s'il choisit une localisation lui-même
 - L'utilisateur soumet le formulaire
- Le système vérifie si le formulaire est valide
- Si le formulaire est valide :
 - Le système enregistre l'annonce en BDD
 - Le système envoie un message de confirmation à l'utilisateur

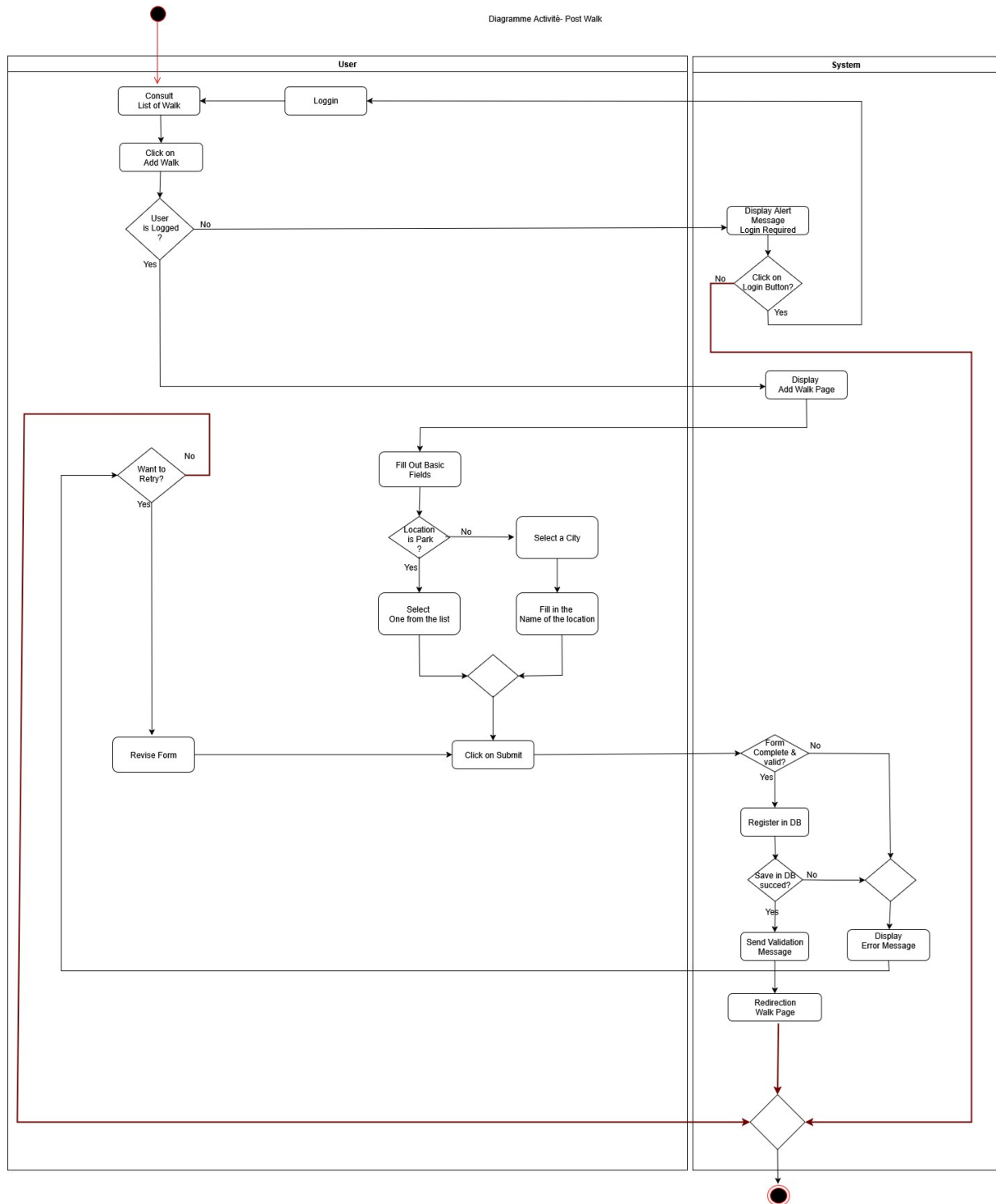
Enchaînements si Erreurs :

Voici les scénarios d'enchaînement si erreur :

- Si l'utilisateur n'est pas connecté
 - Le système demande à l'utilisateur s'il veut se connecter, si oui il est redirigé vers la page de connexion
- Si l'utilisateur a mal rempli le formulaire
 - Le système lui demande de corriger ses erreurs
- Si l'enregistrement en base de données a échoué
 - Le système lui envoie un message d'erreur

Diagramme d'activité

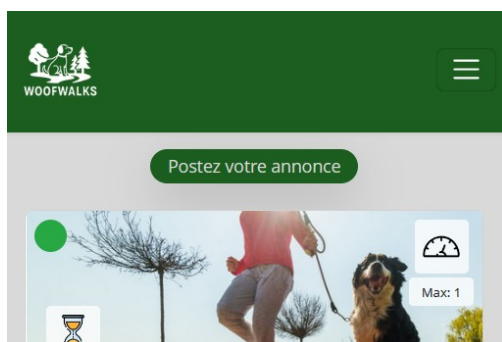
Ce diagramme illustre le processus qu'un utilisateur suit pour poster une annonce pour une balade.



Données

En entrée	Traitement	En sortie	Contrôles
Clic sur Ajouter une balade : - User Id	Vérification que l'utilisateur est connecté	- Affichage du formulaire d'ajout d'une balade ou - Alerte si utilisateur non connecté	L'utilisateur est connecté ?
Clic sur bouton de login :	Redirection vers page de connexion	Page de login	Aucun
Saisie des champs du formulaire	RIEN	RIEN	RIEN
Sélection d'une localisation Parc ou Lieu customisé : - Checkbox	Appel à l'api geo.gouv ou recherche des localisations des parcs pour les afficher à l'utilisateur en Ajax	-Affichage de champs vides qui vont s'auto compléter Ou -Affichage d'une liste de parcs	Lecture du choix de l'utilisateur
Clic pour valider le formulaire : - Formulaire remplie par l'utilisateur	Vérification de la validité du formulaire	-Enregistrement en BDD et message de validation Ou -Message d'erreur	Champs requis sont remplis et valides, conformes aux règles de vérifications programmées

Écran/Affichage



Titre:

Le titre est requis

Description:

La description est requise

Date et heure:

Nombre maximum de participants:

Nombre maximum de participants est requis

Type de lieu:

☐ Lieu personnalisé

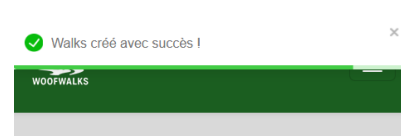
☒ Parc existant

Choisir un parc

-- Sélectionner un parc --

Veillez choisir un parc

Photo: plus.png



5.7.3. Fonctionnalité 3 – Commander un Album Photo

Description

L'utilisateur crée et commande son album photo.

Scénario Nominal :

- L'utilisateur veut commencer à créer son Album.
- L'application cherche les balades en BDD auxquelles l'utilisateur a participé et lui renvoie les Photos de ces balades.
- L'application récupère les Photos
- L'application affiche à l'utilisateur les Photos
- L'utilisateur choisit les photos qu'il veut dans son album
- L'utilisateur confirme sa sélection
- L'application stocke l'album créé et l'ajoute au panier de l'utilisateur
- L'application demande à l'utilisateur s'il veut aller dans son panier
- Si l'utilisateur veut commander immédiatement :
 - L'application le redirige vers le panier
 - L'utilisateur valide son panier
 - L'application récupère les coordonnées de l'utilisateur en BDD
 - L'application demande à l'utilisateur si ses coordonnées sont à choisir en tant que coordonnées de livraison
 - L'utilisateur modifie ou non les coordonnées
 - L'application stocke les coordonnées choisies par l'utilisateur
 - L'application affiche la page de confirmation de commande à l'utilisateur
 - L'utilisateur confirme la commande
 - L'application l'enregistre en BDD
 - L'application demande au service de notification de créer une notification et de l'envoyer
 - Le service de notification exécute la demande, crée une notification et l'envoie à l'utilisateur
 - L'utilisateur effectue la procédure de paiement
 - L'application reçoit le retour de la procédure de paiement et envoie un message de succès à l'utilisateur
 - L'application procède fait appel au PhotoService pour passer commande au prestataire
 - Le service reçoit un message de prise en compte de la part du prestataire
 - L'application reçoit validation de la commande et modifie le statut de la commande

Enchaînements si Erreurs :

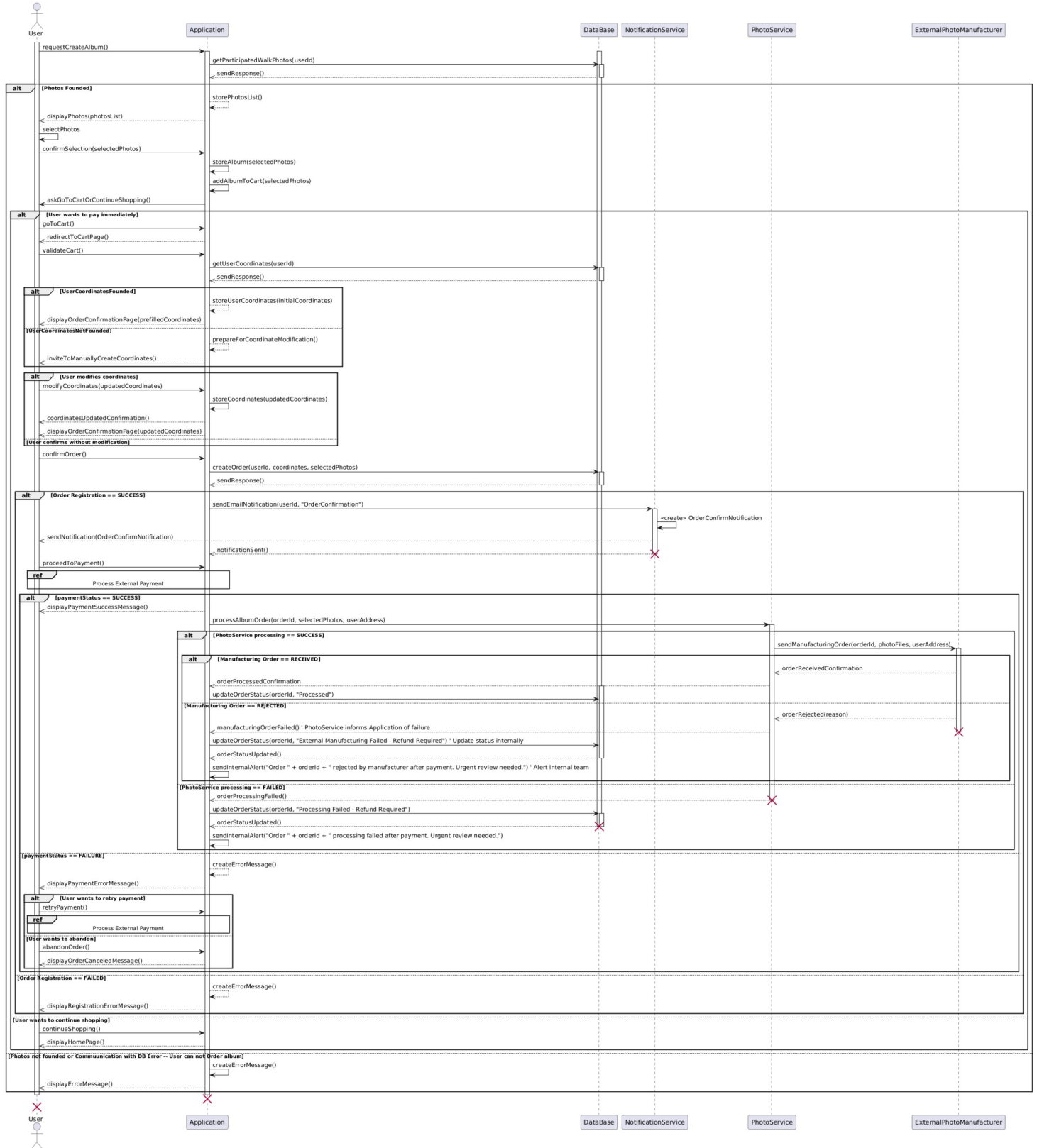
- L'application ne reçoit pas de retour de la BDD lors de la demande de retrouver les photos des balades auxquelles l'utilisateur a participé :
 - L'application crée un message d'erreur et l'affiche à l'utilisateur

- L'enregistrement de la commande ne s'est pas effectué en BDD :
 - L'application crée un message d'erreur et l'envoi à l'utilisateur
- L'application n'a pas retrouvé l'adresse de l'utilisateur :
 - L'application demande à l'utilisateur s'il veut l'insérer manuellement
- Le paiement a échoué :
 - L'application Crée un message d'erreur
 - L'application affiche le message d'erreur à l'utilisateur l'invitant à réessayer
- Le prestataire d'impression de photo signale que la commande est rejetée :
 - Le prestataire informe le service photo que la commande a échoué
 - Le service photo en informe l'application pour qu'elle mette le status de la commande à jour
 - La BDD envoie sa réponse
 - L'application répand une alerte interne
- La demande de l'application au service photo a échoué :
 - Le service photo en informe l'application pour qu'elle mette le status de la commande à jour
 - La BDD envoie sa réponse
 - L'application répand une alerte interne

Diagramme de séquence

Ce diagramme illustre le processus qu'un utilisateur suit pour poster une annonce pour commander un Album Photo

Order Photo Album Process



Données/Traitements

En entrée	Traitement	En sortie	Contrôles
Demande de récupération des photos balades : - Id de l'utilisateur	Recherche en BDD des photos associées aux balades auxquelles il a participé	Liste de photos	L'id de la balade existe ? Des Feedbacks y sont associés ? Des Photos associées aux feedback sont présentes en BDD ?
Confirmation d'une sélection de photos : - Liste de photos - Id de l'utilisateur	Stockage dans l'application de la sélection de photos Ajout dans le panier	Message à l'utilisateur et demande s'il veut aller au panier	
Validation du panier : - Id de l'utilisateur	Récupération des coordonnées de l'utilisateur	Réponse de la BDD	L'utilisateur existe ?
Réponse de la BDD	Stockage dans l'application des coordonnées	Message demandant si l'utilisateur veut les modifier manuellement	Les coordonnées existent ?
Formulaire de validation coordonnées : - Coordonnées Utilisateur	L'application affiche le formulaire destiné à l'enregistrement des coordonnées L'utilisateur les modifie ou non L'application stocke les coordonnées définitives de la commande	Renvoi de l'application vers la page pour confirmer la commande	Le formulaire est complet et l conforme aux règles de programmées ?
Confirmation de la commande : - Commande -Id Utilisateur	L'application enregistre la commande en bdd	L'application reçoit une réponse de la BDD	Les photos et les coordonnées existent ?
Demande au service notification d'informer l'utilisateur que sa commande s'est bien passée : -Id Utilisateur	Le service de notification crée une notification	Envoi d'un email de confirmation à l'utilisateur Envoi de l'information à l'application que l'email a bien été envoyé	L'utilisateur existe ? La commande existe ? Le message a bien été envoyé ?
Paieement	Procédure externe	Procédure externe	Procédure externe
Réponse du process de paiement :	Récupération du status du paiement	Envoi d'un message de succès de paiement ou d'erreur à l'utilisateur	Status du paiement
- Numéro de la commande - Liste de photos - Coordonnées	L'application fait appel au service Photo	Le service Photo tente de passer commande chez le prestataire et répond à l'application	Le numéro de commande, les photos et coordonnées existent ?
Réponse du service photo	Si l'appel au service photo a échoué : la commande n'est pas allée jusqu'au prestataire donc L'application crée et envoie un message d'alerte interne Dans tous les cas : L'application met à jour le status de la commande en BDD	Réponse de la BDD	Échec ou réussite de l'appel au service photo ? Échec ou réussite de l'appel du service photo au prestataire externe

5.7.4. Fonctionnalité 4 – Invitation à poster un feedback

Description

Woofwalks invite l'utilisateur à poster un feedback au sujet de la balade à laquelle il a participé.

Scénario Nominal :

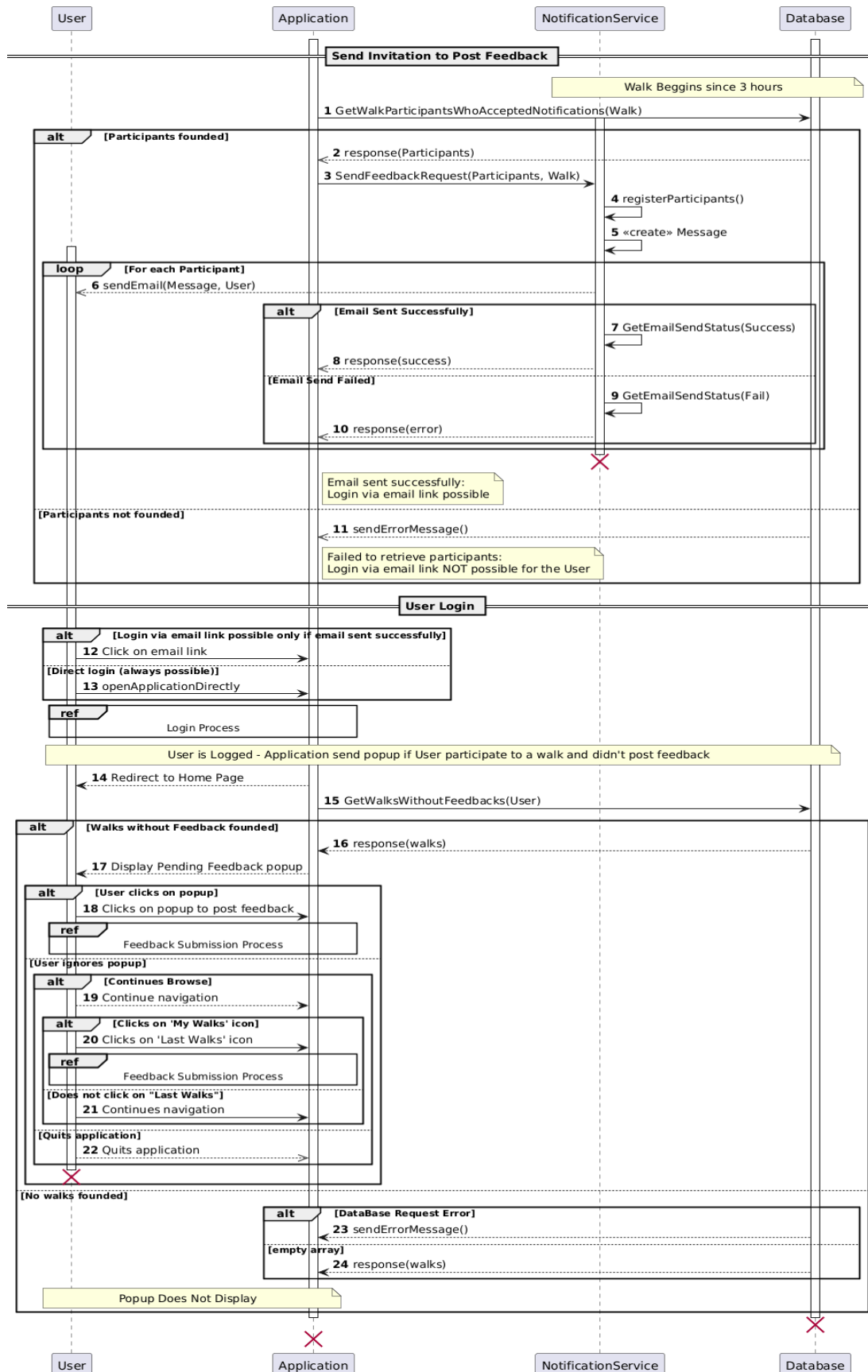
- L'application demande à la BDD de récupérer les participants ayant accepté les notifications pour la marche.
- La BDD lui envoie la liste de participants
- L'application demande au Service de notification au Service de notification d'envoyer une notification aux participants
- Le Service de Notification récupère les participants à notifier et crée un message
- Pour chaque participant, le service de notification envoie un Email
- Le service de notification récupère le statut de l'envoi et notifie à l'application de ce statut
- L'utilisateur va sur l'application via le lien (Il peut aussi accéder à l'entrée sur l'application de manière spontanée, sans être passé par la réception de l'e-mail)
- L'application le dirige vers la page d'accueil
- L'application récupère les balades auxquelles l'utilisateur n'a pas posté de feedback :
- Si la liste des balades trouvées n'est pas vide :
 - L'application lui affiche une popup lui indiquant qu'il n'a pas posté de feedback pour sa dernière balade
 - L'utilisateur clique sur cette popup pour poster un feedback et est redirigé vers la procédure pour poster un feedback
- L'utilisateur peut aussi ignorer la popup et aller sur la liste de ses balades passées pour entamer la procédure de post de feedback

Enchaînements si Erreurs :

- La BDD n'a pas renvoyé de liste de participants :
 - Retour d'un message d'erreur à l'application
- L'e-mail de demande de feedback ne s'est pas envoyé :
 - Le service de notification en informe l'application
- A la connexion de l'utilisateur, la BDD n'a pas réussi à trouver les balades sans Feedback liées à l'utilisateur suite à un plantage :
 - La BDD envoie un message d'erreur à l'application

Diagramme de séquence

Ce diagramme illustre le processus d'envoi d'une invitation à un utilisateur à poster un Feedback, lorsqu'il a fini de participer à une balade



Données/traitement

En entrée	Traitement	En sortie	Contrôles
Requête de l'application (demande de participants ayant accepté les notifications pour la marche)	L'application interroge la BDD pour récupérer la liste des participants.	Liste des participants (ou message d'erreur)	La BDD doit a renvoyé une liste, même si vide
Liste des participants reçue de la BDD	L'application demande au Service de notification d'envoyer une notification. Le Service de notification crée le message et envoie un e-mail à chaque participant.	Statut de l'envoi de la notification (succès/échec par participant)	La notification n'a pas échoué
Clic sur le lien de l'e-mail ou accès spontané à l'application	L'application détecte la connexion de l'utilisateur.	Redirection vers la page d'accueil	
Identifiant de l'utilisateur	L'application interroge la BDD pour récupérer les balades auxquelles l'utilisateur n'a pas encore posté de feedback.	Liste des balades sans feedback (ou message d'erreur)	L'utilisateur a bien réussi la procédure de login
Liste des balades sans feedback	Si la liste n'est pas vide, l'application affiche une popup. Si l'utilisateur clique, il est redirigé vers la procédure de feedback. L'utilisateur peut aussi ignorer la popup et naviguer vers ses balades passées.	Affichage de la popup, redirection vers la page de feedback, ou affichage de la liste des balades passées	La liste des balades n'est pas vide L'utilisateur est bien connecté

5.7.5. Fonctionnalité 5 – Poster un Feedback

Description :

L'utilisateur Post un Feedback pour une balade à laquelle il a participé.

Scénario Nominal :

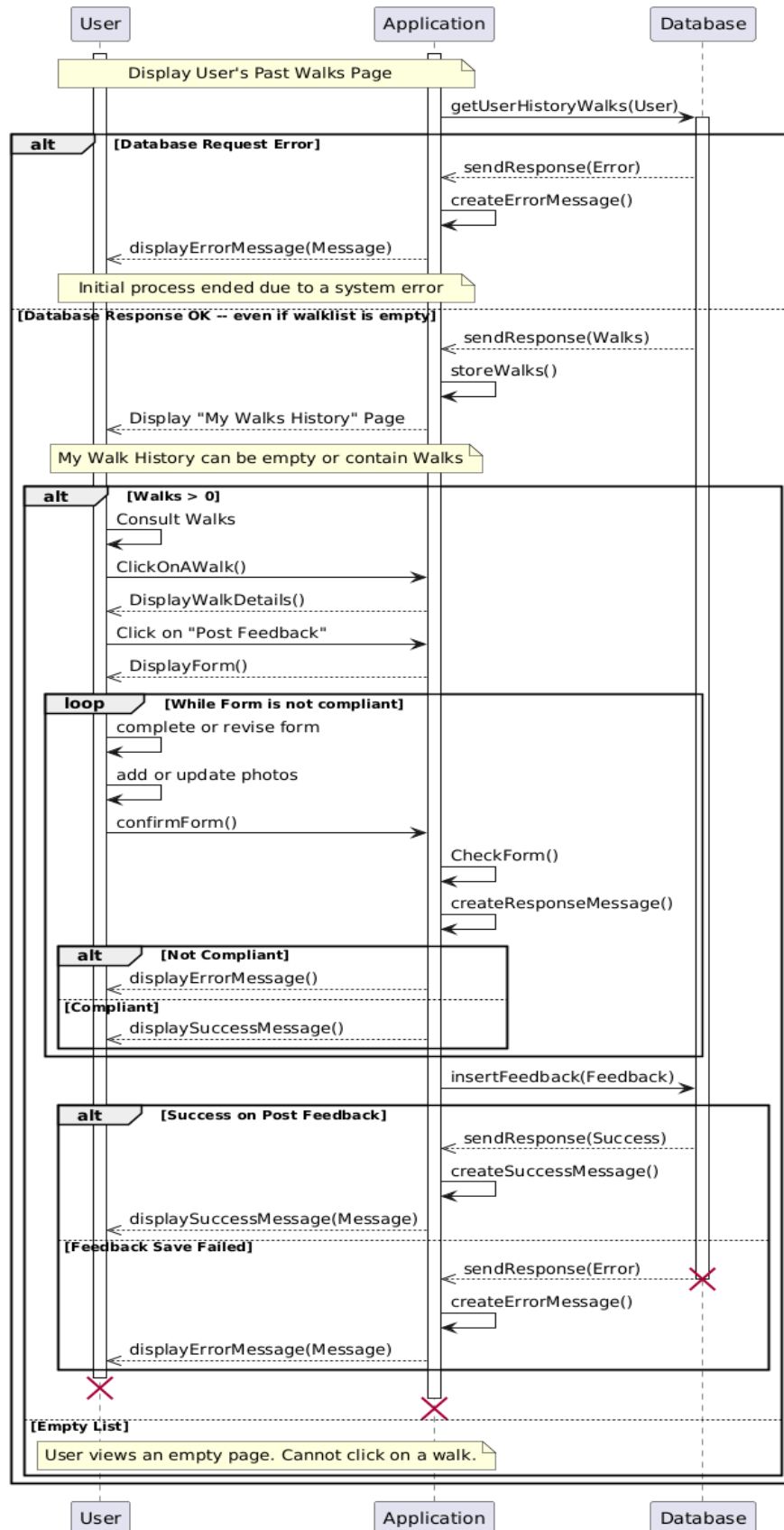
- L'application récupère la liste des balades auxquelles l'utilisateur a participé
- Si les balades ont été trouvées, l'application affiche à l'utilisateur la liste des balades
- L'utilisateur consulte les balades
- L'utilisateur clique sur une balade pour que l'application lui affiche ses détails
- L'application affiche à l'utilisateur les détails de la balade
- L'utilisateur clique sur le bouton pour poster un feedback
- L'application lui affiche le formulaire de post de feedback
- Tant que le formulaire n'est pas complet, l'application va vérifier si les champs sont conformes et bien remplis et éventuellement ajouter ou modifier des photos
- L'utilisateur clique pour poster définitivement le feedback
- L'application enregistre le Feedback en BDD
- Si l'ajout s'est bien effectué : L'application crée un message et l'envoie à l'utilisateur

Enchaînements si Erreurs :

- Si la BDD n'a pas pu récupérer l'historique des balades de l'utilisateur OU Si l'insertion en BDD du Feedback a échoué :
 - Retour d'un message d'erreur à l'application
 - L'application crée un message à envoyer à l'utilisateur
 - L'application envoie le message d'erreur à l'utilisateur

Diagramme de séquence

Ce diagramme illustre le processus de post de Feedback après qu'un utilisateur ait participé à une balade



6. SPÉCIFICATIONS TECHNIQUES

6.1. Référencement

Respect des Standards SEO

Structure HTML sémantique :

- Utilisation de balises HTML5 sémantiques (header, main, section, footer)
- Structure claire et hiérarchisée pour faciliter l'indexation

Balises titres et méta-informations :

Mise en place de Title, Meta description, Open graph et twitter cards

Configuration technique :

- **Robots.txt** : Configuration pour l'indexation des moteurs de recherche
- **Sitemap XML** : Plan du site pour faciliter l'indexation
- **Langue définie** : Balise lang="fr" pour le français

Optimisations spécifiques :

- **URLs SEO-friendly** : Routes descriptives et claires
- **Images optimisées** : Attributs alt descriptifs
- **Responsive design** : Design mobile-first
- **Viewport configuré** : Adaptation aux différents écrans

Environnement technique

Technologies utilisées

Techno	Description	Pourquoi?	
	HTML	Langage de balisage utilisé pour structurer le contenu des pages web.	Permet de créer la structure de base des pages React (via JSX).
	CSS	Langage de style utilisé pour la mise en forme des pages web.	Assure un rendu visuel harmonieux et adapté aux différents écrans.
	JAVASCRIPT	Langage de programmation côté client pour rendre les pages web interactives.	Langage principal pour le développement frontend avec React.
	JSX	Syntaxe combinant HTML et JavaScript, propre à React.	Permet d'écrire des composants React de manière lisible.
	REACT	Bibliothèque JavaScript pour construire des interfaces utilisateur interactives.	Facilite le développement d'une interface fluide et réutilisable.
	PHP	Langage de programmation côté serveur.	Langage supporté par Symfony pour le développement backend.
	SYMFONY	Framework PHP moderne pour développer des applications web robustes.	Structure solide, adaptée à un projet évolutif comme WoofWalks.
	BOOTSTRAP	Framework CSS pour un design responsive.	Assure une interface harmonieuse sur tous les écrans.
	API PLATFORM	Framework basé sur Symfony pour créer des API REST.	Expose automatiquement les entités en API performante.
	JWT AUTHENTICATION	Bundle Symfony pour gérer l'authentification via JWT.	Sécurise les échanges entre le client React et l'API.
	Vich Uploader	Bundle Symfony pour gérer l'upload de fichiers.	Permet aux utilisateurs d'ajouter des images aux annonces.
	Leaflet	Bibliothèque JavaScript pour cartes interactives.	Affiche les annonces sur une carte géographique.
	geo.api.gouv	API publique française pour l'autocomplétion et la géolocalisation.	Facilite la saisie des adresses dans les annonces.
	MySQL	Système de gestion de base de données relationnelle.	Base de données robuste et bien intégrée à Symfony/Doctrine.
	Doctrine	ORM pour PHP/Symfony.	Permet de gérer les entités et les relations en BDD.
	Docker	Plateforme de conteneurisation pour les applications.	Facilite le déploiement et la portabilité du projet.
	Figma	Outil de conception d'interfaces (UI/UX).	Pour la création des maquettes de l'interface WoofWalks.
	Postman	Outil pour tester les API.	Permet de tester les requêtes et analyser les réponses.
	GitHub	Outils de versionnage et d'hébergement de code.	Assure une gestion collaborative du projet.
	PHP My Admin	Interface web permettant de gérer les bases de données.	Permet l'administration des bases de données via une interface graphique.
	VsCode	Éditeur de code multiplateforme.	Environnement de développement principal du projet.

Navigation et accessibilité :

L'application a été mise en production sur **Railway** avec une architecture en deux services indépendants, permettant un déploiement automatique à chaque push sur les repositories frontend et backend.

En raison des contraintes de sécurité CSRF et des jetons XSRF, seules les routes publiques sont navigables. Cette limitation découle de la séparation des domaines entre les deux services Railway, rendant impossible la conservation du système de sécurité JWT avec protection CSRF sans domaine unique.

L'application est compatible avec tous les navigateurs modernes et intègre une conception responsive pour une expérience utilisateur optimale sur tous les appareils.

Services tiers :

- API Géographique (geo.gouv)
- Système d'email (SMTP)
- JWT Authentication (Lexik JWT)

6.2. Sécurité

Cf.Partie 8

Gestion du versioning :

Outil et objectif :

Le projet utilise Git comme système de gestion de versions afin d'assurer :

- Une **traçabilité** des modifications
- Une **collaboration simplifiée**
- Une **stabilité du code en production**

Stratégie de branches :

- **Branche principale :**
 - develop → utilisée comme branche de référence pour les développements en cours.
- **Branches de fonctionnalités** (*feature branches*) :
 - Chaque nouvelle fonctionnalité ou correctif a été développée dans une branche dédiée.

Exemple de nommage : feature/authentication, feature/gestion-annonces, fix/token-expiration.

- **Merges :**
 - Une fois la fonctionnalité terminée et validée, la branche est fusionnée dans develop.

Cette stratégie permet de travailler de manière isolée sans perturber les autres parties du projet.

Commits :

- Utilisation du standard **Conventional Commits** pour un historique clair et structuré

7. RÉALISATIONS

7.1. Exemple 1 : Poster une annonce pour Balade (Composants Métier)

7.1.1. Affichage : *Cf.Diagrammes*

7.1.2. Extrait(s) de code

WalkPage et BtnPostAd

```
const WalksPage = () => {  
  //Passage du context présent  
  const formContext = "walks";  
  return (  
    <>  
      <BtnPostAd  
        formContext={formContext}  
        entitySpecificFields={walkSpecificFields}  
      />  
      <WalkList />  
    </>  
  );  
};  
  
export default WalksPage;
```

```
const BtnPostAd = ({  
  formContext,  
  //Par exemple champs spécifiques à walks  
  entitySpecificFields,  
}) => {  
  const { isAuthenticated } = useAuth();  
  const navigate = useNavigate();  
  
  const [showForm, setShowForm] = useState(false);  
  
  const handleClick = () => {  
    if (!isAuthenticated) {  
      navigate("/login", { replace: true });  
      return;  
    }  
    setShowForm(true);  
  };  
  
  return (  
    <>  
      {showForm ? (  
        <GenericPostAdForm  
          entityType={formContext}  
          entitySpecificFields={entitySpecificFields}  
        />  
      ) : (  
        <button  
          onClick={handleClick}  
          className="button-green btn-success btn-lg shadow-lg rounded-pill px-3 py-1"  
        >  
          Postez votre annonce  
        </button>  
      )}  
    </>  
  );  
};
```

GenericPostAdForm – Le formulaire

```
return (
  <form className="post-ad-form" onSubmit={handleSubmit(onSubmit)}>
    { /* titre */ }
    <div className="form-group"> ...
    </div>
    { /* description */ }
    <div className="form-group"> ...
    </div>
    { /* affiche champs spéciaux */ }
    {entitySpecificFields.fields.map((field) => ( ...
    ))}
    { /* Pour Walks, choix entre parc et loc custom pour les locations + passage des fonctions du useForm*/ }
    {entityType === "walks" ? (
      <WalkLocationSection
        locationType={locationType}
        //Permet de donner le control sur le form au composant enfant
        control={control}
        register={register}
        errors={errors}
      />
    ) : (
      // localisation api gouvernement
      <Controller
        name="locationData"
        control={control}
        defaultValue={{
          city: "",
          street: "",
          latitude: null,
          longitude: null,
          name: "",
        }}
        render={({ field }) => (
          <SelectLocationForm
            value={field.value}
            onLocationDataChange={field.onChange}
          />
        )}
      />
    )}
  />
)

<PhotoForm photo={photo} onChange={handleFileChange} />

<button type="submit" className='button-green' disabled={isSubmitting}>
  {isSubmitting ? "En cours..." : `Créer ${entityType}`}
</button>
```


GenericPostForm (onSubmit) , getLocationId et appel à api pour upload de photo

```
const onSubmit = async (data) => {

  if (!photo) {
    alert("Veuillez sélectionner une photo !");
    return;
  }
  setIsSubmitting(true);

  // SANITIZATION
  const sanitizedData = { ...
  };
  // Vérif que locationData existe avant de Sanitize
  if (data.locationData) { ...
  }
  //récupération Id de la localisation
  const locationId = await getLocationId(sanitizedData);
  if (!locationId) {
    alert("La location doit être spécifiée.");
    setIsSubmitting(false);
    return;
  }
  try {
    const photoFormData = new FormData();
    photoFormData.append("file", photo);
    const photoData = await uploadPhoto(photoFormData);
    const photoId = photoData.id;

    const formattedDateTime = new Date(sanitizedData.datetime).toISOString();

    const entityData = {
      ...sanitizedData,
      date: formattedDateTime,
      photo: photoId,
      location: locationId,
      is_custom_location: sanitizedData.use_custom_location === "custom",
    };

    await postGenericAd(entityData, entityType);

    reset({
      title: "",
      description: "",
      location: "",
      datetime: "",
      ...entitySpecificFields.initialValues,
    });
  }
};
```

```
const getLocationId = async (data) => {
  if (data.use_custom_location === "custom") {
    const locationData = await createLocation(data.locationData);
    return parseInt(locationData["@id"].split("/").pop());
  } else if (data.use_custom_location === "park") {
    return parseInt(data.park_location_id);
  }
  return null;
};
```

```
export const uploadPhoto = async (data) => {
  const response = await api.post("/main_photo", data);
  return response.data;
};
```

Appel au contrôleur custom et contrôleur custom

```
export const postGenericAd = async (data, entityType) => {
  try {
    const response = await api.post(`${entityType}custom`, data);
    const result = response.data;

    toast.success(
      `${entityType.charAt(0).toUpperCase() + entityType.slice(1)} créé avec succès !`,
      {
        position: "top-right",
        autoClose: 3000,
        hideProgressBar: false,
        closeOnClick: true,
        pauseOnHover: true,
        draggable: true,
        progress: undefined,
      }
    );
    return result;
  } catch (error) {
    console.error("Erreur gérée par l'intercepteur.");
    throw error;
  }
};
```

Contrôleur:

```
class WalkController extends AbstractController {
  #[Route('/api/walkscustom', name: 'create_walk', methods: ['POST'])]
  public function createWalk(
    Request $request,
    WalkWithChatCreationInterface $walkCreator,
    WalkValidationService $validationService
  ): JsonResponse {

    $user = $this->getUser();
    if (!$user) {
      return new JsonResponse(['error' => 'Authentication required'], 401);
    }
    $data = json_decode($request->getContent(), true);

    // Validation centralisée par le Service validation
    $validationResult = $validationService->validateWalkData($data);

    if (!$validationResult->isValid()) {
      return new JsonResponse(['errors' => $validationResult->getErrors()], 400);
    }
    //Passe par Interface pour appeler Service de Création
    $walk = $walkCreator->createWalkWithChat($data);

    if (!$walk) {
      return new JsonResponse(['error' => 'Failed to create walk (invalid data or dependencies)'], 400);
    }

    return new JsonResponse(['message' => 'Walk created successfully'], 201);
  }
}
```

Interface, Orchestrateur et Services

```
interface WalkWithChatCreationInterface {
    public function createWalkWithChat(array $data): ?Walk;
    //Possibilité de creer createWalkOnly ou addChatToExistingWalk
}
```

```
class WalkAndChatOrchestrator implements WalkWithChatCreationInterface {
    private WalkCreationService $walkCreationService;
    private ChatCreationService $chatCreationService;

    public function __construct(
        WalkCreationService $walkCreationService,
        ChatCreationService $chatCreationService
    ) {
        $this->walkCreationService = $walkCreationService;
        $this->chatCreationService = $chatCreationService;
    }

    public function createWalkWithChat(array $data): ?Walk {
        // Création de la Walk
        $walk = $this->walkCreationService->createWalk($data);

        if (!$walk) {
            return null;
        }
        //Création du Chat associé
        $this->chatCreationService->createChatForWalk($walk);

        return $walk;
    }
}
```

```
public function createWalk(array $data): ?Walk {
    // Validation de la Data par le Service validation
    $validationResult = $this->validationService->validateWalkData($data);
    if (!$validationResult->isValid()) {
        return null;
    }

    try {
        $datetime = new \DateTime($data['datetime']);
    } catch (\Exception $e) {
        return null;
    }

    $photo = $this->photoRepository->find($data['photo']);
    if (!$photo) {
        return null;
    }

    $location = $this->locationRepository->find($data['location']);
    $isCustomLocation = isset($data['is_custom_location']) && $data['is_custom_location'] === true;
    $creator = $this->security->getUser();

    if (!$creator instanceof User) {
        return null;
    }

    $walk = new Walk();
    $walk->setTitle($data['title']);
    $walk->setDescription($data['description']);
    $walk->setMainPhoto($photo);
    $walk->setDate($datetime);
    $walk->setMaxParticipants($data['max_participants'] ?? 0);
    $walk->setLocation($location);
    $walk->setCreator($creator);
    $walk->setIsCustomLocation($isCustomLocation);
    $walk->addParticipant($creator);

    $this->entityManager->persist($walk);
    $this->entityManager->flush();

    return $walk;
}
```

```
class ChatCreationService {
    private EntityManagerInterface $entityManager;

    public function __construct(EntityManagerInterface $entityManager) {
        $this->entityManager = $entityManager;
    }

    public function createChatForWalk(Walk $walk): Chat {
        $chat = new Chat();
        $chat->setWalk($walk);

        // Établir la relation bidirectionnelle
        $walk->setChat($chat);

        $this->entityManager->persist($chat);
        $this->entityManager->flush();

        return $chat;
    }
}
```

7.1.3. Argumentation :

Lors de la création d'une Walk, l'utilisateur interagit avec un composant *GenericPostAdForm* qui gère de manière générique la soumission d'annonces pour différentes entités du système. Ce composant reçoit en propriétés les champs spécifiques à l'entité Walk et affiche successivement les champs génériques (titre, description, date) puis les champs spécialisés (location, nombre de participants maximum).

Gestion de la localisation : Pour les entités de type **Walk**, le composant *WalkLocationSection* permet de choisir entre un **Parc** existant (stocké en base de données) ou une localisation personnalisée. Cette section intègre un *SelectLocationForm* qui utilise l'API géographique du gouvernement français (geo.gouv) pour la sélection des coordonnées.

Traitement des données : Une fois les champs renseignés, l'identifiant de localisation est récupéré via la fonction *getLocationId()*. Si un Parc est sélectionné, l'identifiant est immédiatement disponible. Dans le cas d'une localisation personnalisée, la fonction *createLocation()* crée une nouvelle entrée en base de données et retourne l'identifiant généré.

Upload et soumission : L'image associée est traitée par le service *uploadPhoto* via un contrôleur dédié. L'ensemble des données est ensuite transmis au service *postGenericAd* qui utilise un endpoint générique réutilisable (*api/{entityType}custom*) pour différentes entités.

Validation et persistance : Le contrôleur backend vérifie l'authentification de l'utilisateur, valide les données via le service *WalkValidationService*, puis procède à la création de la **Walk** via l'interface *WalkWithChatCreationInterface*, paramétré dans *services.yaml* pour correspondre à *WalAndChatOrchestrator*. Ce dernier implémente l'Interface puis définit la fonction *createWalkWithChat()* (appelé par le contrôleur) qui utilise *WalkCreationService* et *ChatCreationService* pour insérer la data permettant de créer une Walk et son Chat.

7.2. Exemple 2 : Enregistrement d'un Utilisateur (Composants Métier)

7.2.1. Affichage

The image displays three sequential screenshots of the 'Inscription' (Registration) form in the WOOFWALKS application. Each screenshot features a dark green header with the WOOFWALKS logo and a hamburger menu icon.

First Screenshot (Initial State): The form is titled 'Inscription'. It contains input fields for 'E-mail:' (filled with 'herberichs.sarah@gmail.com'), 'Nom d'utilisateur:' (filled with 'Herberichs33Sh'), and 'Mot de passe:' (filled with ten dots). Below the password field, a red error message states: 'Le mot de passe doit contenir au moins une majuscule, un chiffre et un caractère spécial.' There is a 'Confirmer le mot de passe:' field with ten dots. Below it, another red message says: 'Les mots de passe ne correspondent pas.' At the bottom, there is a checked checkbox for 'J'accepte les Conditions Générales de Vente' and a green 'S'inscrire' button.

Second Screenshot (Successful Registration): The form is titled 'Inscription'. A green success message is displayed: 'Inscription réussie, un email de vérification vous a été envoyé !'. The input fields are empty. The 'Confirmer le mot de passe:' field is also empty. The checkbox for 'Conditions Générales de Vente' is checked, and the green 'S'inscrire' button is visible at the bottom.

Third Screenshot (Duplicate Email Error): The form is titled 'Inscription'. A red error message is shown at the top: 'Détails: Cet email est déjà utilisé pour un autre compte.' Below this, the input fields are filled with the same data as in the first screenshot. The 'Confirmer le mot de passe:' field is also filled with ten dots. The checkbox for 'Conditions Générales de Vente' is checked, and the green 'S'inscrire' button is visible at the bottom.

7.2.2. Extrait(s) de code

HOOK **useUserRegistration** , qui gère la **logique métier** : Sanitize, contrôle les champs, envoi la requête d'enregistrement en BDD via userService.js , gère les retours d'erreurs, réinitialise le formulaire.

```
export const useUserRegistration = () => {
  const [username, setUsername] = useState("");
  const [email, setEmail] = useState("");
  const [password, setPassword] = useState("");
  const [confirmPassword, setConfirmPassword] = useState("");
  const [cgv, setCgv] = useState(false);
  const [errors, setErrors] = useState({});
  const [registrationSuccess, setRegistrationSuccess] = useState(false);
  const [loading, setLoading] = useState(false);

  const handleSubmit = async (e) => {
    e.preventDefault();
    setErrors({});
    setRegistrationSuccess(false);
    setLoading(true);

    // SANITIZATION + Validation
    const safeUsername = DOMPurify.sanitize(username.trim(), { ALLOWED_TAGS: [], ALLOWED_ATTR: [] });
    const safeEmail = DOMPurify.sanitize(email.trim(), { ALLOWED_TAGS: [], ALLOWED_ATTR: [] });
    const validationErrors = {};

    if (!safeEmail) {
      validationErrors.email = "L'e-mail est requis.";
    } else if (!/^\S+@\S+\.\S+/.test(safeEmail)) {
      validationErrors.email = "L'e-mail n'est pas valide.";
    } else if (safeEmail.length > 180) {
      validationErrors.email = "L'e-mail ne peut pas dépasser 180 caractères.";
    }

    if (!safeUsername) {
      validationErrors.username = "Le nom d'utilisateur est requis.";
    } else if (safeUsername.length > 100) {
      validationErrors.username = "Le nom d'utilisateur ne peut pas dépasser 100 caractères.";
    }

    if (!password) {
      validationErrors.password = "Le mot de passe est requis.";
    } else if (password.length < 6) {
      validationErrors.password = "Le mot de passe doit contenir au moins 6 caractères.";
    } else if (!/[A-Z]/.test(password) || !/[0-9]/.test(password) || !/[!@#%&*()_~+=<>?]/.test(password)) {
      validationErrors.password = "Le mot de passe doit contenir au moins une majuscule, un chiffre et un caractère spécial.";
    }
  }
}
```

```

try {
  await registerUser({
    email: safeEmail,
    username: safeUsername,
    plainPassword: password,
    cgvAccepted: cgv,
  });
  setUsername("");
  setEmail("");
  setPassword("");
  setConfirmPassword("");
  setErrors({});
  setRegistrationSuccess(true);
} catch (err) {
  console.error("Erreur d'inscription :", err);
  const serverErrors = {};
  const data = err.response?.data;
  if (data) {
    if (data.violations && Array.isArray(data.violations)) {
      data.violations.forEach((violation) => {
        const field = violation.propertyPath || "general";
        serverErrors[field] = violation.message;
      });
    } else if (data.detail) {
      const match = data.detail.match(/^([a-zA-Z0-9_]+):s*(.*)$/);
      if (match && match.length === 3) {
        const field = match[1];
        const message = match[2];
        serverErrors[field] = message;
      } else {
        serverErrors.general = data.detail;
      }
    } else if (data.message) {
      serverErrors.general = data.message;
    } else {
      serverErrors.general = "Une erreur est survenue lors de l'inscription.";
    }
  } else {
    if (err.message.includes("Duplicate entry")) {
      serverErrors.email = "Cet email est déjà utilisé pour un autre compte.";
    } else {
      serverErrors.general = "Erreur de connexion au serveur. Veuillez réessayer.";
    }
  }
  setErrors(serverErrors);
} finally {
  setLoading(false);
}
};

return {
  username,
  setUsername,
  email,
  setEmail,
  password,
  setPassword,
  confirmPassword,
  setConfirmPassword,
  cgv,
  setCgv,
  errors,
  registrationSuccess,
  loading,
  handleSubmit,
};

```

Service registerUser qui tente l'enregistrement en BDD

```
import api from "../api";

export const registerUser = async ({ email, username, plainPassword, cgvAccepted }) => {
  const response = await api.post("/users", {
    email,
    username,
    plainPassword,
    cgvAccepted,
  });
  return response.data;
};
```

Form UseRegistrationForm qui gère l'affichage

```
import { useUserRegistration } from '../hooks/useUserRegistration';
import './UserRegistration.css';

const UserRegistrationForm = () => {
  // Utilise le hook pour accéder à toute la logique
  const {
    username,
    setUsername,
    email,
    setEmail,
    password,
    setPassword,
    confirmPassword,
    setConfirmPassword,
    cgv,
    setCgv,
    errors,
    registrationSuccess,
    loading,
    handleSubmit,
  } = useUserRegistration();

  return (
    <form className="user-registration-form" onSubmit={handleSubmit}>
      <h2>Inscription</h2>

      {registrationSuccess && (
        <p className="success-message">
          Inscription réussie, un email de vérification vous a été envoyé !
        </p>
      )}
      {errors.general && <p className="error-message">{errors.general}</p>}

      <div className="form-group">
        <label htmlFor="email">E-mail:</label>
        <input
          type="email"
          id="email"
          value={email}
          onChange={(e) => setEmail(e.target.value)}
          required
        />
        {errors.email && <p className="error-message">{errors.email}</p>}
      </div>
    </form>
  );
};
```


UserRegistrationSubscriber

```
// va se déclencher après désérialisation mais avant validation et persistance
public static function getSubscribedEvents(): array {
    return [
        KernelEvents::VIEW => ['onPreRegisterUser', 200],
    ];
}

public function onPreRegisterUser(ViewEvent $event): void {
    $user = $event->getControllerResult();
    $method = $event->getRequest()->getMethod();

    // On s'intéresse uniquement aux requêtes POST sur l'entité User (pour l'inscription)
    if (!$user instanceof User || $method !== 'POST') {
        return;
    }
    // Récupère l'e-mail du nouvel utilisateur qui tente de s'inscrire
    $email = $user->getEmail();
    if (empty($email)) {
        return;
    }
    // Recherche un utilisateur existant avec cet e-mail
    $existingUser = $this->userRepository->findOneBy(['email' => $email]);
    if ($existingUser) {
        $expirationTime = (new \DateTimeImmutable())->modify('-24 hours');
        // Si l'utilisateur est en BDD mais n'a pas validé son compte même s'il a reçu son mail - on supprime
        if (!$existingUser->isVerified() &&
            $existingUser->getConfirmationRequestedAt() !== null &&
            $existingUser->getConfirmationRequestedAt() < $expirationTime) {

            $this->entityManager->remove($existingUser);
            $this->entityManager->flush();
            return;
        }
    }

    $event->setResponse(new JsonResponse([
        'detail' => 'Cet email est déjà utilisé pour un autre compte.'
    ], 400));
    return;
}
```

UserDataPersister

```
public function process(mixed $data, Operation $operation, array $uriVariables = [], array $context = []): mixed {
    if (!$data instanceof User) {
        return $data;
    }

    if ($data->getPlainPassword()) {
        // Gestion du Hashage
        $hashedPassword = $this->passwordHasher->hashPassword($data, $data->getPlainPassword());
        $data->setPassword($hashedPassword);
        $data->setPlainPassword(null);
    }

    if ($operation->getMethod() === 'POST' && $data->getId() === null) {
        $this->logger->info('Frontend URL utilisée pour le mail : ' . $this->frontendUrl);
        // Génération token
        $token = Uuid::v4()->toRfc4122();
        $data->setConfirmationToken($token);

        $data->setIsVerified(false);

        $data->setConfirmationRequestedAt(new \DateTimeImmutable());
        $data->setRoles(['ROLE_USER']);

        $frontendUrl = $this->frontendUrl;
        $confirmationUrl = $frontendUrl . '/confirm-email?token=' . $data->getConfirmationToken();
        // Envoi le mail
        $email = (new Email())
            ->from('admin@terrashare.fr')
            ->to($data->getEmail())
            ->subject('Confirmez votre adresse email')
            ->html("<p>Merci pour votre inscription. Cliquez sur le lien suivant pour confirmer votre adresse : <a href='{$confirmationUrl}'>Confir");

        $this->mailer->send($email);
    }
}
```

7.2.3. Argumentation :

FRONTEND :

1 - Composant d'affichage (UserRegistrationForm)

Le composant **UserRegistrationForm** constitue la couche de présentation.

Il utilise le hook personnalisé **useUserRegistration** pour accéder à l'état et aux fonctions de gestion du formulaire.

Ce composant se contente :

- d'afficher les champs de saisie
- de gérer les interactions utilisateur
- de présenter les messages d'erreur ou de succès

2 - Hook personnalisé (useUserRegistration)

Le hook encapsule toute la logique métier côté client :

- **Gestion des états** : username, email, password, confirmPassword, cgV, errors, loading
- **Validation côté client** : vérification des formats, longueurs, mots depasse
- **Sanitisation** : utilisation de *DOMPurify* pour nettoyer les entrées utilisateur
- **Gestion des erreurs** : traitement des erreurs de validation et des réponses serveur
- **Appel API** : communication avec le service registerUser

3 - Service de communication (userService)

Le service **registerUser** gère la communication HTTP avec l'API backend, en encapsulant la logique de requête et de gestion des réponses.

BACKEND :

1 - Event Subscriber (UserRegistrationSubscriber)

Intercepte les événements de création d'utilisateur avant la persistance :

- Vérification d'unicité : contrôle si l'email existe déjà
- Gestion des comptes non vérifiés : suppression des comptes non confirmés après 24h
- Validation métier : application des règles spécifiques au domaine

2 - Data Persister (UserDataPersister)

Gère la persistance des données utilisateur :

- Hachage du mot de passe : transformation du mot de passe en clair en hash sécurisé
- Génération du token de confirmation : création d'un token unique pour la vérification email
- Persistance en base : sauvegarde de l'entité User via l'EntityManager

3 - Validation des contraintes

Les contraintes de validation sont définies directement sur l'entité User :

- Contraintes de format : email valide, longueur des champs
- Contraintes métier : acceptation des CGV obligatoire
- Contraintes de sécurité : complexité du mot de passe

7.3. Exemple 3 : (Composants d'accès aux données) : Voir Ci-dessus, utilisation des Repositories et de l' entity manager

8. ÉLÉMENTS DE SÉCURITÉ DE L'APPLICATION

Création de compte et confirmation par email

La création de compte est sécurisée pour prévenir les enregistrements frauduleux et les spams. L'objectif est de s'assurer que l'utilisateur est bien propriétaire de l'adresse email fournie.

Hachage du mot de passe : Lors de la création du compte, le mot de passe est haché à l'aide d'un algorithme sécurisé, via l'interface **PasswordAuthenticatedUserInterface** de Symfony. Aucune donnée en clair n'est conservée

Validation par token : Un **token unique** est généré et envoyé à l'utilisateur sous forme de lien. Ce token, à usage unique et d'une durée limitée (24 heures), est obligatoire pour valider le compte. La connexion est impossible tant que le compte n'est pas activé

Protection contre les comptes fantômes : Les comptes non confirmés dans les 24 heures sont automatiquement supprimés, évitant l'accumulation de données inutiles et la création de comptes fantômes

L'authentification

Elle repose sur la **gestion de deux jetons différents**, chacun avec un rôle spécifique.

Authentification JWT : La connexion se fait via l'URL `/api/login_check` et est gérée par le bundle **LexikJWTAuthenticationBundle**. En cas de succès, le backend génère deux jetons :

Un **JWT** de courte durée (par exemple, 15 minutes) utilisé pour authentifier l'utilisateur à chaque requête. Ce jeton prouve son identité

Un **refresh token** de longue durée, utilisé uniquement pour prolonger la session

Sécurité des cookies : Les deux jetons sont stockés dans des **cookies HttpOnly**, ce qui les rend inaccessibles au code JavaScript du navigateur et prévient le vol de session via une attaque XSS. Ces cookies sont également configurés avec `withSecure`, forçant leur transmission via une connexion **HTTPS sécurisée**, et avec `withSameSite='lax'`, ce qui prévient les attaques de type CSRF en restreignant leur envoi depuis des sites tiers

Contrôle et renouvellement des sessions : À chaque requête sécurisée, le JWT de l'utilisateur est lu et vérifié. S'il a expiré, le système utilise le refresh token (toujours valide) pour générer un nouveau JWT. Cette mécanique assure que l'utilisateur n'a pas à se reconnecter constamment, tout en garantissant que le jeton principal est de courte durée, limitant les risques en cas de vol

L'implémentation du **contrôle d'authentification** niveau front se fait dans un **contexte React d'authentification**, qui centralise l'état de l'utilisateur (connecté ou non), fournit les fonctions de login et logout, et gère automatiquement le rafraîchissement des tokens via Axios pour que tous les composants enfants puissent accéder à ces informations et comportements sans répéter la logique

Sécurité Générale :

- **Pas de stockage côté client** : Aucune donnée sensible comme les mots de passe ou les tokens n'est stockée dans le **localStorage** ou le **sessionStorage** du navigateur, ce qui réduit considérablement les risques de vol de données
- Configuration d'un .ini désactivant l'exposition de la version de PHP :

```
root@8e3bf540c03e:/usr/local/etc/php/conf.d# echo "expose_php = Off" > 99-custom.ini
root@8e3bf540c03e:/usr/local/etc/php/conf.d#
```

- Création d'un web_profiler désactivant les outils de débogage en production

```
voofwalks_back > config > packages > prod > web_profiler.yaml.bak
1 web_profiler:
2   toolbar: false
3   intercept_redirects: false
4
```

- Utilisation de Nelmio Security Bundle et configuration des Entêtes pour la production via le fichier .default.conf, Ce fichier gère la redirection, les chemins des certificats SSL, les redirections si fichiers introuvables et ajoute toutes les entêtes nécessaires à sécuriser l'application :
 - **add_header X-Content-Type-Options nosniff** : Éviter de changer le de contenu type d'un fichier
 - **add_header Strict-Transport-Security** : Éviter une **attaque de l'homme du milieu (MitM)**
 - **add_header Content-Security-Policy**:Protège des XSS
 - **fastcgi_hide_header X-Powered-By**: masque la version de PHP
 - **server_tokens off** : Masque la version de Nginx

Gestion des rôles et des accès :

Les accès aux ressources sont strictement contrôlés pour garantir que seuls les utilisateurs autorisés peuvent effectuer des actions.

- **Définition des rôles** : Les rôles, comme **ROLE_USER** ou **ROLE_ADMIN**, sont définis dans le fichier **security.yaml** de Symfony.
- **Contrôle d'accès** : Des **firewalls** et des règles **access_control** sécurisent les routes. Les routes publiques sont accessibles à tous, tandis que d'autres fonctionnalités (comme la modification d'un profil ou la création d'une annonce) ne sont disponibles que pour les utilisateurs authentifiés possédant le rôle approprié.
- **Vérification double** : Les contrôles de permissions sont effectués à la fois côté frontend (pour masquer les fonctionnalités non accessibles à l'utilisateur) et côté backend.

Validation des données :

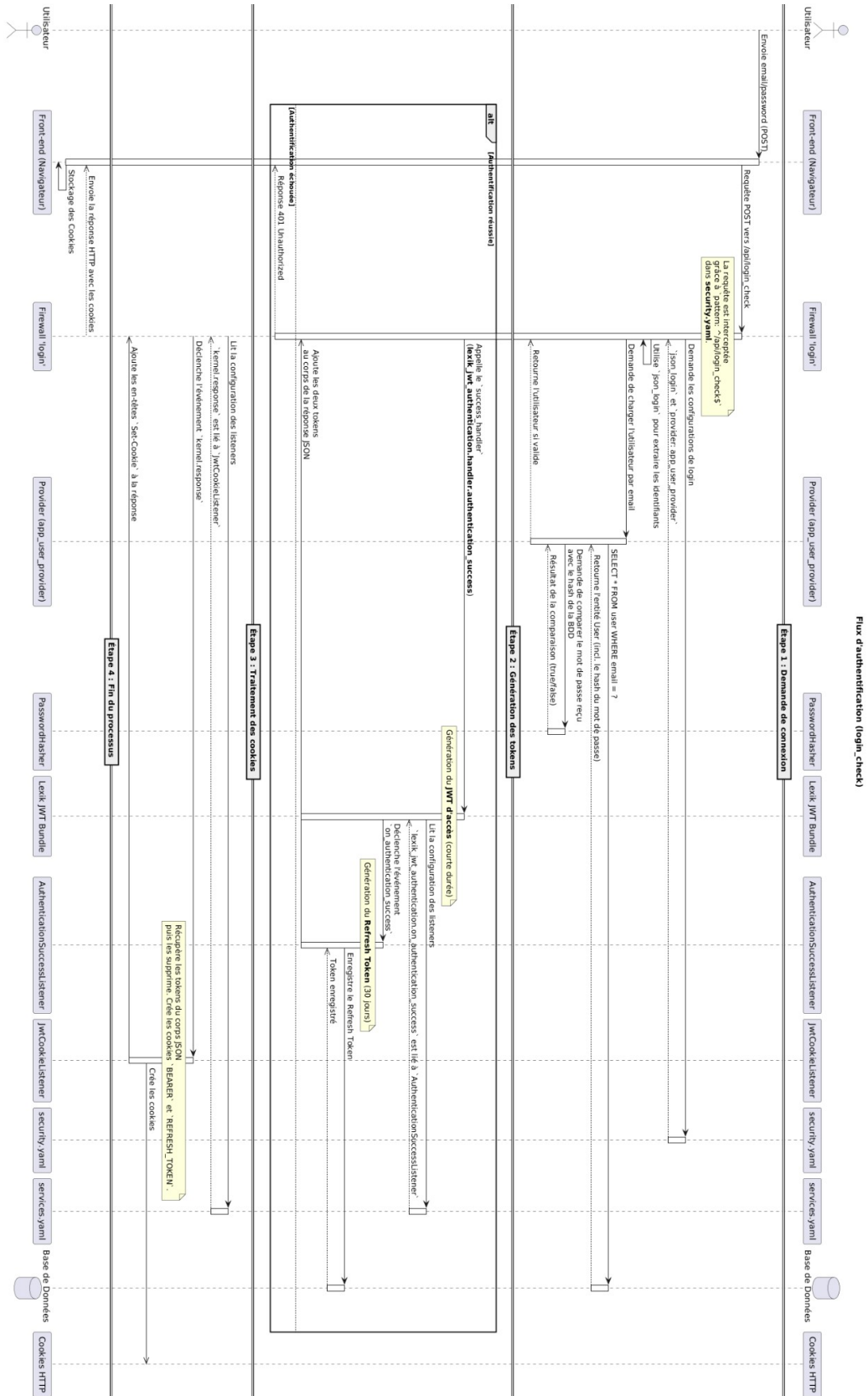
Vérification double : Les contrôles de données entrantes sont effectués à la fois côté frontend et côté backend.

La stratégie de validation des données :

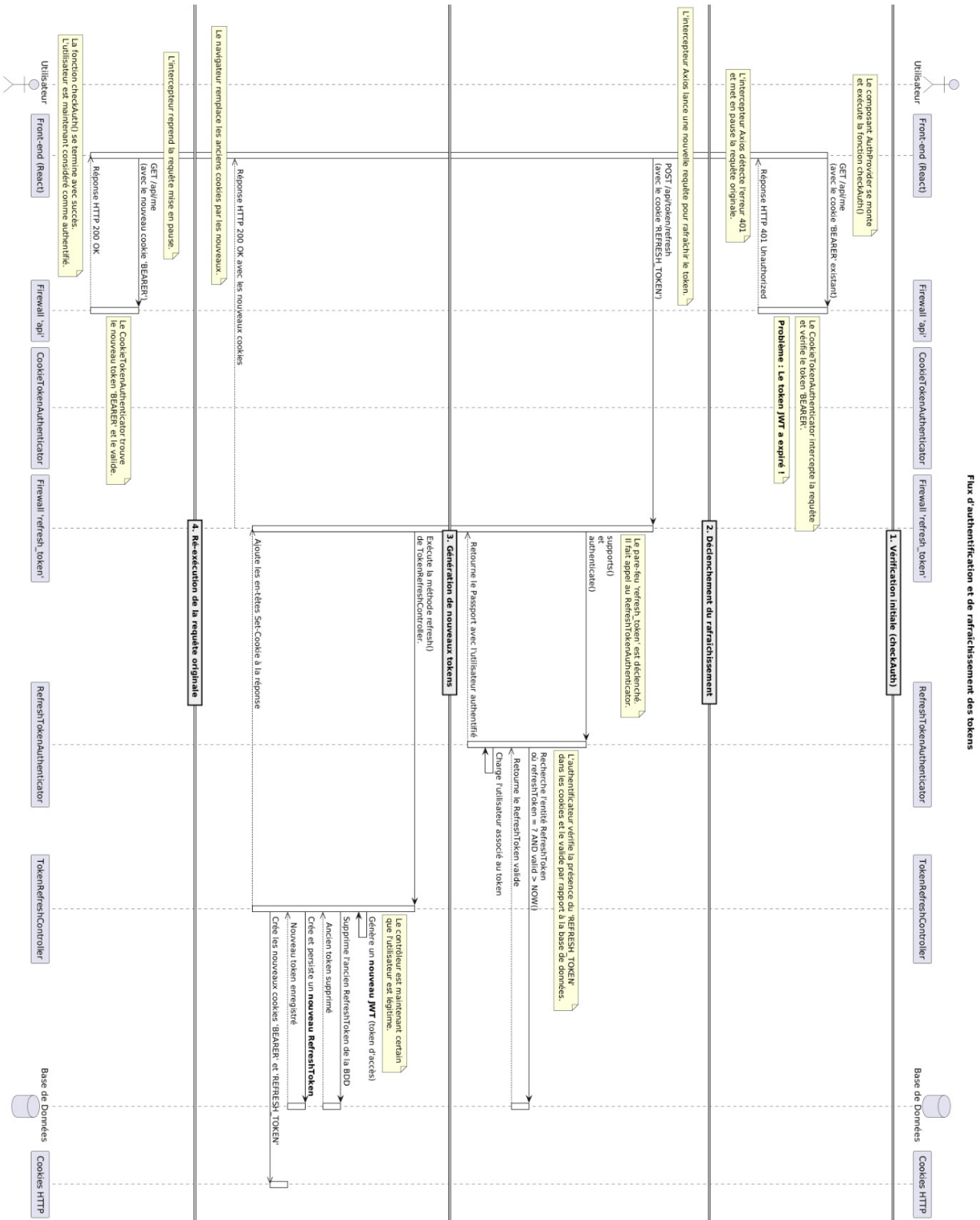
Pour garantir l'intégrité des données, une stratégie de validation à trois niveaux a été mise en place

1. **Validation côté client (Frontend)** : Des vérifications de base en JavaScript sont utilisées pour s'assurer que les champs requis ne sont pas vides et que les formats de données sont corrects avant d'envoyer la requête à l'API.
2. **Validation côté serveur (Backend)** : C'est la couche de sécurité essentielle. Le composant **Validator de Symfony** bloque les requêtes invalides. Les contraintes sont définies directement sur les entités (par exemple, `#[Assert\Email]` pour un email valide ou `#[Assert\Length]` pour la longueur d'un mot de passe).
3. **Contraintes de la base de données (SGBD)** : La dernière couche de sécurité est assurée par des contraintes dans le schéma de la base de données (comme la contrainte UNIQUE sur le champ email), garantissant ainsi qu'il n'y a aucun doublon.

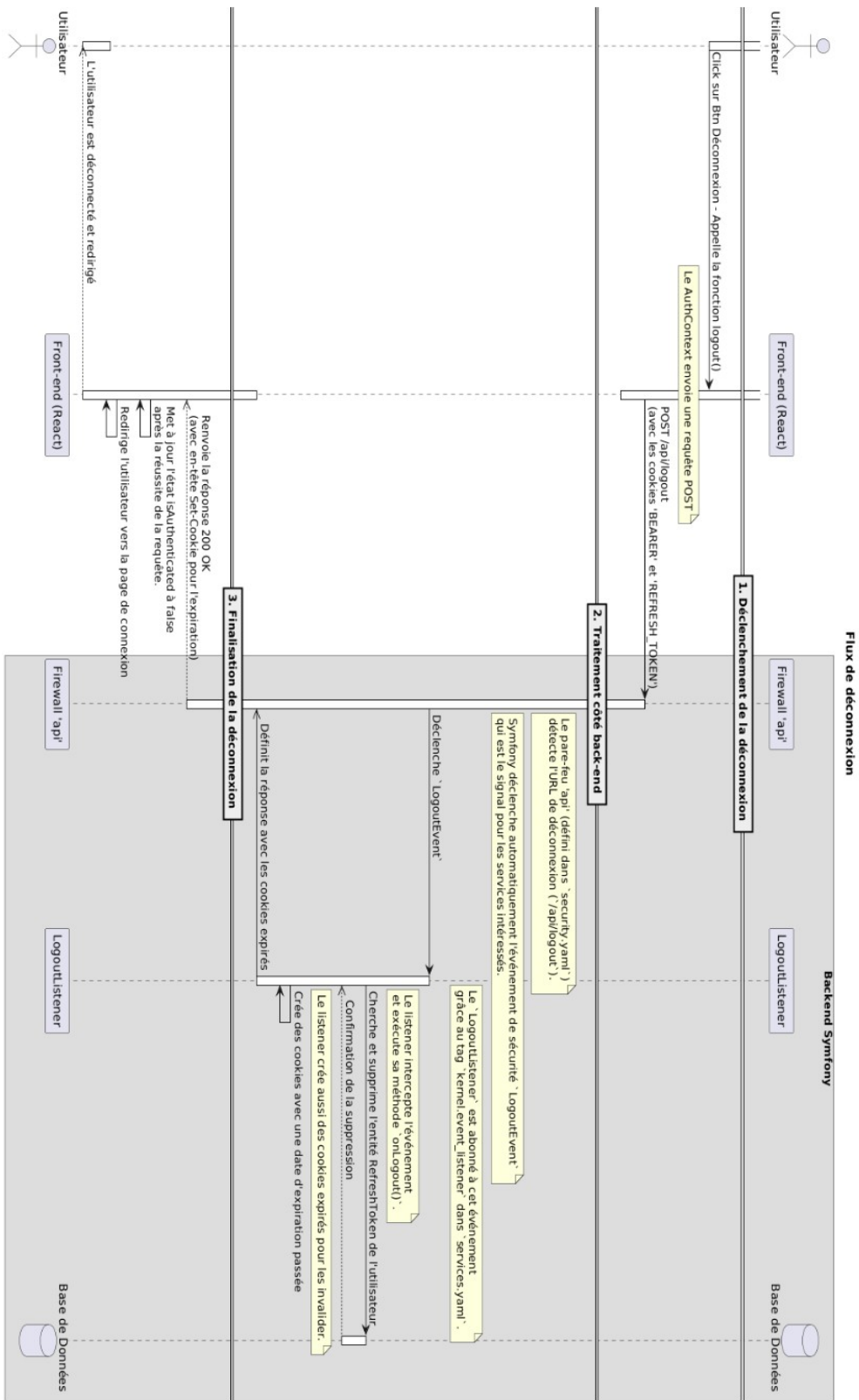
AUTHENTIFICATION



Contrôle d'authentification et Refresh



Déconnexion



9. PLAN DE TESTS

ID	Partie Testée	Description	Étapes	Résultat Attendu	Type de test
UT-001	Entité User	Tester les accesseurs et mutateurs de base (email, username, password, roles).	1. Instancier un nouvel utilisateur.-2. Appeler les mutateurs pour définir les valeurs.-3. Vérifier que les accesseurs renvoient les bonnes valeurs.	Les méthodes getEmail(), getUsername(), etc. doivent renvoyer les valeurs attendues.	Unitaire
UT-002	Entité User	Tester la relation "créateur de balade".	1. Créer une nouvelle instance de User et de Walk 2. Appeler user->addCreatedWalk(\$walk) et user->removeCreatedWalk(\$walk).	La relation bidirectionnelle doit être correctement gérée (la balade doit être ajoutée/retirée de la collection de l'utilisateur, et l'utilisateur doit être défini/supprimé comme créateur de la balade).	Unitaire
UT-003	Entité User	Tester la relation "participant à une balade".	1. Créer une nouvelle instance de User et de Walk.-2. Appeler user->addParticipatedWalk(\$walk) et user->removeParticipatedWalk(\$walk).	La relation bidirectionnelle doit être correctement gérée (l'utilisateur doit être ajouté/retiré de la balade, et la balade doit être ajoutée/retirée de la collection de l'utilisateur).	Unitaire
WT-001	Entité Walk	Vérifier la création de l'entité avec ses valeurs par défaut.	1. Instancier une nouvelle balade (new Walk()).	L'entité doit être une instance de Walk, et les champs createdAt, updatedAt, participants et isCustomLocation doivent avoir les valeurs par défaut attendues.	Unitaire
WT-002	Entité Walk	Tester les accesseurs et mutateurs (getters/setters).	1. Instancier une nouvelle balade.-2. Appeler les mutateurs (setTitle, setDescription, etc.) avec des valeurs.-3. Vérifier que les accesseurs renvoient les bonnes valeurs.	Les accesseurs doivent renvoyer les valeurs définies par les mutateurs.	Unitaire
WT-003	Entité Walk	Tester l'ajout et la suppression de participants.	1. Créer une balade et un utilisateur.-2. Appeler walk->addParticipant() et walk->removeParticipant().	La balade doit contenir l'utilisateur dans sa collection de participants après l'ajout, et ne plus le contenir après la suppression.	Unitaire
ECCT-001	Contrôleur Confirmation	Tenter de confirmer un email avec un token invalide.	1. Appeler la méthode confirmEmail() avec un token qui n'existe pas.	Une réponse JSON avec un statut 400 et le message "Lien de confirmation invalide ou déjà utilisé."	Unitaire
ECCT-002	Contrôleur Confirmation	Tenter de confirmer un email déjà vérifié.	1. Appeler la méthode confirmEmail() avec un token valide mais pour un utilisateur déjà vérifié.	Une réponse JSON avec un statut 200 et le message "Votre adresse email est déjà vérifiée."	Unitaire
ECCT-003	Contrôleur Confirmation	Confirmer un email avec succès.	1. Appeler la méthode confirmEmail() avec un token valide.-2. Les mocks simulent un utilisateur valide.	Une réponse JSON avec un statut 200 et le message "Adresse email confirmée..."	Unitaire
WCT-001	Contrôleur Walk	Créer une balade avec succès.	1. Simuler une requête avec des données valides.-2. Le mock du service de création retourne une balade valide.	Une réponse avec un statut 201 et un message de succès.	Unitaire
WCT-002	Contrôleur Walk	Tenter de créer une balade sans tous les champs requis.	1. Simuler une requête avec des données incomplètes.	Une réponse avec un statut 400 et le message "Missing required fields".	Unitaire
WCT-003	Contrôleur Walk	Tenter de créer une balade si le service échoue.	1. Simuler une requête avec des données valides.-2. Le mock du service de création retourne null.	Une réponse avec un statut 400 et le message "Failed to create walk".	Unitaire
WCFT-001	Création de balade	Créer une balade via l'API avec succès.	1. Créer un client HTTP.-2. Générer un token JWT pour l'utilisateur.-3. Envoyer une requête POST à l'API avec les données et le token.	Un statut HTTP 201 et une réponse JSON de succès, avec le message "Walk created successfully".	Fonctionnel
US-FT-01	User	Tenter d'accéder à la liste de tous les utilisateurs en tant qu'administrateur.	1. Créer un utilisateur avec le rôle ROLE_ADMIN. 2. Se connecter en tant que cet utilisateur. 3. Envoyer une requête GET à /api/users.	Une réponse HTTP 200 OK (succès) et un contenu JSON valide.	Fonctionnel
US-FT-02	User	Tenter d'accéder à la liste de tous les utilisateurs en tant qu'utilisateur normal.	1. Créer un utilisateur normal. 2. Se connecter en tant que cet utilisateur. 3. Envoyer une requête GET à /api/users.	Une réponse HTTP 403 Forbidden (accès refusé).	Fonctionnel
US-FT-03	User	Tenter d'accéder à son propre profil en tant qu'utilisateur normal.	1. Créer un utilisateur normal. 2. Générer un token JWT pour cet utilisateur. 3. Envoyer une requête GET à /api/users/{id} en utilisant l'ID de l'utilisateur, et inclure le token dans l'en-tête Authorization.	Une réponse HTTP 200 OK (succès).	Fonctionnel
US-FT-04	User	Tenter d'accéder au profil d'un autre utilisateur en tant qu'utilisateur normal.	1. Créer deux utilisateurs normaux (user1 et user2). 2. Générer un token JWT pour user1. 3. Envoyer une requête GET à /api/users/{id} en utilisant l'ID de user2, et inclure le token de user1 dans l'en-tête Authorization.	Une réponse HTTP 403 Forbidden (accès refusé).	Fonctionnel

10. JEU D'ESSAI DE LA FONCTIONNALITÉ LA PLUS REPRÉSENTATIVE

10.1. Fonctionnalité testée

L'accès au profil utilisateur a été retenue pour faire présenter le jeu d'essai. Cette fonctionnalité, accessible par une API REST et dont les droits sont encadrés directement par les expressions de sécurité Symfony dans l'entité User, permet à un utilisateur authentifié de consulter les informations de son propre profil. Elle est également protégée par un système d'authentification par token JWT. Seuls les utilisateurs connectés ont le droit de faire cette requête. Elle est essentielle pour garantir que les utilisateurs peuvent accéder à leurs données personnelles de manière sécurisée et que personne ne peut usurper leur identité pour consulter ou modifier leur profil.

```
#[Get(  
    // POUR GET /api/users/{id} : ADMIN OU L'UTILISATEUR LUI-MÊME  
    security: "is_granted('ROLE_ADMIN') or (is_granted('IS_AUTHENTICATED_FULLY') and object == user)",  
    securityMessage: "Accès refusé. Vous devez être administrateur ou consulter votre propre profil."  
)]
```

10.2. Description des scenarios

Jeu d'essai	Description	Résultat attendu
US-FT-01	Accès à la liste des utilisateurs en tant qu'administrateur.	Le système doit retourner un code de statut 200 OK et un contenu JSON valide.
US-FT-02	Accès à la liste des utilisateurs en tant qu'utilisateur normal.	Le système doit refuser l'accès et retourner un code de statut 403 Forbidden.
US-FT-03	Accès à son propre profil en tant qu'utilisateur normal.	Le système doit retourner un code de statut 200 OK.
US-FT-04	Accès au profil d'un autre utilisateur en tant qu'utilisateur normal.	Le système doit refuser l'accès et retourner un code de statut 403 Forbidden.

10.3. Résultats des tests

Jeu d'essai	Résultat attendu	Résultat obtenu
US-FT-01	Réponse HTTP 200 OK	Réussi. La requête a bien retourné un statut 200 avec la liste de tous les utilisateurs.
US-FT-02	Réponse HTTP 403 Forbidden	Réussi. La requête a été correctement refusée avec un statut 403.
US-FT-03	Réponse HTTP 200 OK	Réussi. La requête a bien retourné un statut 200 et les données du profil de l'utilisateur authentifié.
US-FT-04	Réponse HTTP 403 Forbidden	Réussi. La requête a été correctement refusée avec un statut 403.

10.4. Conclusion

Les tests de performance que j'ai réalisés sur l'application ont été concluants et ont atteint les objectifs définis. J'ai pu valider la stabilité du système.

```
public function testAdminCanSeeAllUsers(): void {
    // Création d'un admin et d'un utilisateur normal
    $admin = $this->createUser('admin_' . uniqid() . '@example.com', ['ROLE_ADMIN']);
    $user = $this->createUser('user_' . uniqid() . '@example.com');
    $user = $this->createUser('user_' . uniqid() . '@example.com');

    // Forcer la connexion de l'admin
    $this->client->loginUser($admin);

    // Faire la requête protégée en HTTPS
    $this->client->request(
        'GET',
        '/api/users',
        [],
        [],
        ['HTTP_ACCEPT' => 'application/json', 'HTTPS' => true]
    );

    $this->assertResponseIsSuccessful();
    $this->assertJson($this->client->getResponse()->getContent());
}

public function testRegularUserCannotSeeAllUsers(): void {
    $user = $this->createUser('user_' . uniqid() . '@example.com');

    $this->client->loginUser($user);

    $this->client->request(
        'GET',
        '/api/users',
        [],
        [],
        ['HTTP_ACCEPT' => 'application/json', 'HTTPS' => true]
    );

    // Accès refusé pour un user normal
    $this->assertResponseStatusCodeSame(403);
}
```

```
public function testUserCanSeeOwnProfile(): void {
    $user = $this->createUser('user_' . uniqid() . '@example.com');

    $this->client->loginUser($user);

    $jwtManager = self::getContainer()->get('lexik_jwt_authentication.jwt_manager');
    $token = $jwtManager->create($user);

    $this->client->request(
        'GET',
        '/api/users/' . $user->getId(),
        [],
        [],
        [
            'HTTP_ACCEPT' => 'application/json',
            'HTTPS' => true,
            'HTTP_AUTHORIZATION' => 'Bearer ' . $token
        ]
    );

    $this->assertResponseIsSuccessful();
}

public function testUserCannotSeeOtherProfile(): void {
    $user1 = $this->createUser('user_' . uniqid() . '@example.com');
    $user2 = $this->createUser('user_' . uniqid() . '@example.com');

    $this->client->loginUser($user1);

    $jwtManager = self::getContainer()->get('lexik_jwt_authentication.jwt_manager');
    $token = $jwtManager->create($user1);

    $this->client->request(
        'GET',
        '/api/users/' . $user2->getId(),
        [],
        [],
        [
            'HTTP_ACCEPT' => 'application/json',
            'HTTPS' => true,
            'HTTP_AUTHORIZATION' => 'Bearer ' . $token
        ]
    );

    $this->assertResponseStatusCodeSame(403);
}
```

11. VEILLE SUR LES VULNÉRABILITÉS DE SÉCURITÉ

Je porte une attention particulière à la sécurité de mon application. J'ai effectué des tests passifs basés sur les recommandations de l'OWASP Top 10, afin de vérifier que je n'avais pas omis de gérer certaines failles critiques.

Pour rester informée, je suis abonnée à plusieurs newsletters spécialisées dans PHP, ainsi qu'aux actualités quotidiennes de DailyNews. Je consulte également régulièrement The Hacker News, ce qui me permet de suivre les nouvelles vulnérabilités et tendances en cybersécurité.

Je m'engage aussi à maintenir mon application à jour en appliquant systématiquement les correctifs de sécurité sur les frameworks, bundles et dépendances que j'utilise, afin de réduire les risques d'exploitation.

Cette veille régulière, combinée aux bonnes pratiques que je mets en œuvre, me permet de renforcer la sécurité de mon projet au quotidien.